

Implied Volatility in One Straight Line: Machine Precision at Vector-Hardware Throughput

Wolfgang Schadner*

September 15, 2026

Abstract

Recovering Black–Scholes implied volatility from an option price is demanded at machine precision and at vectorized-hardware throughput. We present an inverter that replaces branch-rich iterative refinement by lane-uniform straight-line evaluation. In the inverse’s intrinsic coordinates the expansion about the small-moneyness line is exact. Every coefficient beyond one tabulated function of one variable is rational, and a conformal re-expansion carries the same rows into the deep wing. One straight-line kernel with a single branch and a 63-term table answers all but three quotes in thirty thousand of a market-realistic S&P 500 feed, four routed charts serve the rest. Every kernel is a fixed sequence of fused multiply-adds shared by the scalar and SIMD paths, so batched results are bit-identical across instruction sets, compilers and the CUDA port. Against a 40-digit oracle the worst relative error is 5.7×10^{-16} , below the reference on every validation set. Against the reference at its release flags, the batch path inverts the feed at 6.7 (AVX-512) and 3.8 (AVX2) times its rate. On a datacenter GPU the kernel inverts a region-partitioned feed at 0.032 nanoseconds per quote; on the same card and quotes the nearest published table method is six times slower and ten orders of magnitude less accurate.

1 Introduction

The implied volatility of a European option is the value of the volatility parameter that equates the Black–Scholes price to an observed market price. Because it is the coordinate in which option prices are quoted, compared, interpolated, and risk-managed, its computation is, in the words of Jäckel [13], at once one of the most frequently executed and one of the least documented numerical tasks in derivatives modelling. A single recalibration of a volatility surface [8], a local-volatility bootstrap, or a Monte Carlo pricing grid can require millions of inversions, and each must be accurate. An inverter that is correct only to a few digits injects noise into every downstream Greek and every calibrated parameter. The workload shape matters as much as the count. Production systems rarely invert one quote at a time. They invert a strip of strikes at one maturity, a whole surface, or a dense grid, and on modern hardware the difference between a method that processes such batches eight lanes at a time and one that cannot is a larger factor than any remaining difference in single-quote cost. That batch axis, not scalar latency, is where the present work aims.

Two largely separate literatures address the problem. The first seeks explicit, closed-form approximations to the inverse map. Early formulas supplied Newton seeds and near-the-money estimates [4, 6, 20] and later work widened accuracy and domain. Li [19] fits rational functions of moneyness and maturity, Stefanica and Radoičić [29] derive an explicit formula with a uniform relative-error bound, and Xia and Cui [31] an explicit series inversion. These are fast and naturally vectorizable, but they hold only on a bounded region of the input plane and only

*University of Liechtenstein, `wolfgang.schadner@uni.li`

to modest precision. The second literature refines an initial guess to full machine precision. The reference method here is Jäckel [14]’s *Let’s Be Rational*, which combines a four-branch rational seed with two third-order Householder iterations on a carefully reformulated normalized Black function, and attains the last representable bit throughout the domain. Its accuracy is not in question. The cost is that a branch-rich refinement, fixed in count but not in path, is difficult to vectorize, and its natural unit of work is a single scalar quote. A third, smaller strand precomputes the inverse map itself. Glau et al. [9] interpolate the implied-volatility surface globally in Chebyshev polynomials, the approach closest in spirit to ours. Matić et al. [22] fit a polynomial table by a PDE method for batch-oriented use, with public C++/OpenCL code, but target an accuracy of 10^{-7} , eight orders of magnitude short of the last bit, and reach it with a coefficient file of 46 megabytes.

The gap this leaves is a method that reaches machine precision *and* vectorizes over the whole feasible domain, so that the batched workloads which dominate production (an entire book of quotes, or a dense calibration grid) run at hardware throughput rather than at scalar-iteration latency. We close that gap in two stages. The first treats inversion not as “seed and refine” but as *routed evaluation with fixed-count local refinements*. Two regimes are pure table or series evaluations, the other two finish with a short *fixed* number of correction steps evaluated without branches, so that every lane executes the same instruction sequence. The reference solver is also fixed-count. The distinction that matters for vector hardware is branch-rich refinement against lane-uniform straight-line arithmetic, and only the book kernel of the second stage needs no refinement at all. The feasible region is partitioned into four regimes, each carrying an approximation in the coordinate natural to it. The interior is a bivariate Chebyshev table in the transformed variable $W = h^2/2w$, whose cell geometry and polynomial degree follow a design principle fixed by the location of the inverse map’s branch points and confirmed cell-by-cell by the fitted degrees. The three endpoint charts are a matched small-moneyness expansion built on the exact at-the-money kernel, an absolutely convergent large-volatility ladder, and an asymptotically peeled deep-wing inversion. A central structural result is that the special functions of all four charts are one two-parameter family, so the entire construction rests on a single evaluator plus the inverse error function. Every quote is routed into its regime by integer operations on its IEEE-754 bit fields, and evaluated by branchless kernels written so that the scalar entry point and the SIMD batch drivers execute the identical sequence of fused multiply-adds per quote. The batch result is therefore bit-for-bit identical to the scalar result, which we treat as a correctness invariant and verify exhaustively. The second stage removes the routing from the book altogether. In the inverse’s intrinsic coordinates the small-moneyness chart admits an exact recurrence in which every coefficient beyond one tabulated function of a single variable is a rational number (Section 5), and the same recurrence, re-expanded in a conformal variable adapted to the nearest singularity, extends across the interior into the deep wing. One straight-line kernel with a 63-term table therefore covers all but three quotes in thirty thousand of the traded book, and the routed charts stand behind it for those three and for the far larger feasible domain a traded book never visits. Enforcing the bit-identity invariant requires a compiler discipline (Section 6) that we believe is not widely appreciated. Under GCC’s default floating-point contraction the same header can compile to two functions that differ in the last bit within one program.

Against an independent high-precision oracle the inverter attains a worst observed relative error in volatility of 5.7×10^{-16} on the validation sets, 9.2×10^{-16} for the routed charts alone (Section 7), below *Let’s Be Rational* on every set and holding machine precision in the two extreme regimes where the reference measurably degrades, and every representable input pair receives either a result of that validated accuracy class or an explicitly flagged non-result. We also ask what a *realistic* workload exercises. Routing an entire year of tradeable S&P 500 index option quotes through the classifier shows the small-moneyness chart carrying 92% of that book, the tabulated chart 7% and the deep wing 1% (Section 7). That concentration is not a nuisance

but a lever. The book kernel of Section 5 turns it into one straight line of arithmetic. Timed in the same binary and the same loop as the reference’s official public implementation, built at its author’s release flags, the kernel’s batch path inverts the market feed at 6.7 times the reference’s rate on AVX-512, and its scalar entry, one quote at a time on the same build, is faster than the reference as well. The AVX2 figures, the routed charts’ own drivers and the warm-restart streaming regime are in Section 7. On tradeable quotes all solvers considered reach their design precision, so the case for the method on real data is throughput and determinism, with the accuracy advantage held in reserve for the extreme inputs a robust deployment must also handle. The same fixed-count kernels, mirrored as CUDA device code, carry the determinism claim across architectures. One NVIDIA H100 returns bitwise-identical doubles for the full market mix, at 0.032 ns per quote with the book kernel on a feed partitioned by its intrinsic coordinate (Section 7). The same section runs the nearest published relative of the design, the table method of Matic’ et al. [22], from its authors’ code on the same card and the same quotes. It is six times slower under its own convention and ten orders of magnitude less accurate on the traded feed, while its single-threaded scalar evaluation is the one figure it wins.

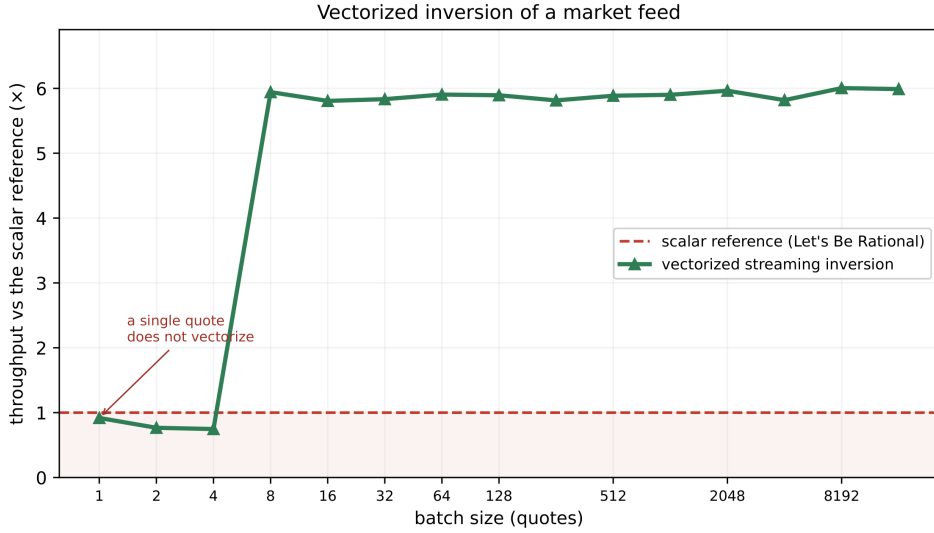


Figure 1: Why vectorization is the point, on real data. Re-inverting a 2024 S&P 500 book from the previous snapshot, the vectorized warm-restart driver of the routed charts (Section 6) sustains about five times the per-quote rate of the scalar reference solver at its release flags (six against the build of Table 5), flat in batch size once a batch fills one SIMD register. The step at eight quotes is the width of an AVX-512 register. A batch of fewer than eight quotes cannot fill one, so the driver runs those quotes through its scalar tail, the same kernel one lane at a time, at the scalar reference’s speed or slightly below it; from eight upward every quote travels in a full register, and the per-quote cost drops by a factor of eight at once rather than gradually. On a four-wide AVX2 build the step sits at four. This is a *deployment-mode* comparison, the warm driver starts from each node’s previous root, under its few-ULP contract (Section 6). The like-for-like cold full-accuracy figures on the same feed, against the release build, are 2.2–4.3 times for the routed charts and 6.7 and 3.8 times for the book kernel of Section 5, which needs no warm start to exceed the streaming rate (Table 4). Throughput is measured against the reference in the same session, so the host clock cancels.

The paper is organized as follows. Section 2 states the method in brief. Section 3 fixes the normalized problem. Section 4 constructs the tabulated chart and the branch-point argument that sizes it, then the endpoint charts, their unification, and the accurate-forward evaluation. Section 5 derives the intrinsic coordinates, proves the rational row structure (Proposition 1), and builds the book kernel from it. Section 6 describes routing and the scalar/SIMD bit-identical

implementation. Section 7 reports accuracy and performance, and Section 8 concludes.

2 The method at a glance

The method has two phases. An offline phase, run once, fits and freezes every coefficient of the tabulated chart and the endpoint charts. A runtime phase then routes each quote into its regime by integer tests on its IEEE-754 bit fields and evaluates it with a fixed, branch-free operation count. Algorithm 1 states both phases as a high-level summary. The construction is developed in Sections 4 and 6 and detailed in Appendices B and C.

Algorithm 1 The routed inverter at a glance

Offline (run once, detailed in Appendix C). From the branch-point principle of Section 4.2, fix the cell geometry. Fit the far-moneyness Chebyshev table, the B^{-1} , V_2 , V_4 , wing and seam polynomials against 40-digit oracles. Gate every cell at 10^{-15} relative in σ . Freeze all coefficients as hexadecimal doubles.

Runtime (per batch of quotes (h_i, c_i)).

- 1: **book kernel** (Section 5): for every quote form $a = \log(1 + \expm1(h)/c)$ and $A_0 = a G(a)$ from one 63-term table; if $a < 2\pi$ and $h \leq 0.35 a$ return $v = t S(t^2, A_0^2)$ with $t = h/A_0$ and S the exact rows; if $a \geq 2\pi$ and h within the wing reach return the same rows in the conformal variable q ; on a traded book this answers all but one quote in ten thousand, straight-line and iteration-free
 - 2: **routed charts**, for the remainder: classify each quote by IEEE-754 bit fields and two frozen seam polynomials into NEAR / FAR / WING / UPPER (Section 4.5), listed in decreasing share of a real book. No price is evaluated to route
 - 3: bucket quotes by chart (counting sort), or, when a sampled probe shows the small-moneyness share is high, evaluate NEAR speculatively on all lanes and defer only the minority
 - 4: **for** each bucket, eight (AVX-512) or four (AVX2) quotes per SIMD register **do**
 - 5: NEAR: $A = B^{-1}(c/\expm1(h))$, matched expansion in $s = h/A$; or, in the intrinsic form of Section 5, $A_0 = a G(a)$, $t = h/A_0$, $v = t S(t^2, A_0^2)$ with S exact
 - 6: FAR: bivariate Clenshaw sweep in (x_h, x_λ) , $w = h^2/(2W)$
 - 7: WING: peeled log-price equation, table-seeded, exactly three Newton steps
 - 8: UPPER: rational seed + ladder + exactly three Householder steps
 - 9: **end for**
 - 10: scatter results to input order. Every kernel is a branchless fused-multiply-add chain shared with the scalar entry, so the batch output is bit-identical to scalar
-

3 The normalized inversion problem

We work throughout with the normalized out-of-the-money (OTM) Black price [3, 23]. Writing $h = |\log(K/F)| \geq 0$ for the absolute log-moneyness, $w = \sigma^2 T$ for the total variance and $v = \sqrt{w} = \sigma\sqrt{T}$ for the total standard deviation, the normalized OTM price is

$$c = \Phi\left(\frac{v}{2} - \frac{h}{v}\right) - e^h \Phi\left(-\frac{v}{2} - \frac{h}{v}\right) \in (0, 1), \quad (1)$$

with Φ the standard normal cumulative distribution function. Calls and puts map to the same problem by the put–call symmetry $h \mapsto -h$, so it suffices to treat $h \geq 0$, and the inversion problem is to recover w (equivalently σ) from a given pair (h, c) . The reduction from raw option data deserves an exact statement, since (1) is written for the out-of-the-money *call* ($K \geq F$). An in-the-money option first maps to its out-of-the-money twin by put–call parity, an out-of-the-money *put* ($K < F$) is the dual call with F and K exchanged, and the surviving premium is normalized by $\min(F, K)$. The resulting (h, c) satisfies (1). The wrapper of Section 6 implements

exactly this map. It is the same reduction used by Jäckel [14], save that the normalizer is $\min(F, K)$ rather than $\sqrt{FK}$, a choice that matters in the deep wing (Section 7).

All of the analysis below flows from a single integral representation of (1). Differentiating the Black price with respect to the total standard deviation and integrating back gives the exact form

$$c = C(h, v) = k e^{h/2} \int_0^v e^{-t^2/8} e^{-h^2/(2t^2)} dt, \quad k = \frac{1}{\sqrt{2\pi}}, \quad C(h, \infty) = 1, \quad (2)$$

in which the integrand is the product of a Gaussian factor $e^{-t^2/8}$, regular at $t = \infty$, and a boundary-layer factor $e^{-h^2/(2t^2)}$ that vanishes to all orders at $t = 0$. The two factors are the two endpoints of the inversion problem. The layer at $t = 0$ governs the small-price wing, and the regular tail governs the large-volatility limit $c \rightarrow 1$. The interior, where neither factor simplifies, is where a table is the natural device. Section 4.2 treats the interior and Sections 4.1–4.3 the two endpoints.

The map (1) is analytic and strictly monotone in v , but it is numerically treacherous at both ends of the price range. As $v \rightarrow 0$ the OTM price collapses like $c \sim \varphi(h/v) v/h$, so a plain Newton iteration on $c - C(h, v)$ has a vanishing derivative and loses all its significant digits. As $v \rightarrow \infty$, $c \rightarrow 1$ and the two terms of (1) cancel. Robust inversion therefore requires working in a variable in which the map is well conditioned. The organizing object is the endpoint factorization of (1). With the exponential scale variable

$$W = \frac{h^2}{2w} = \frac{h^2}{2v^2}, \quad (3)$$

one has the exact representation $c = \gamma(h) e^{-W} W^{-3/2} H(W)$, in which $\gamma(h) = \frac{h}{2} e^{h/2}/(2\sqrt{\pi})$, and H is an elementary, error-function-free convergent integral with $H(W) \sim 1 - (a + \beta)/W + \dots$ for a fixed pair $(a, \beta) = (3/2, h^2/16)$. This factorization is the deep-wing form of the generalized-inverse-Gaussian quantile representation of the Black map established in Schadner [27], where the normalized OTM price is identified as a generalized-inverse-Gaussian distribution in the variance. Viewed as a function of a complex price, the inverse map continues analytically onto a ring-shaped domain (an *annulus*) in the c -plane bounded by two circles, one passing through each of the branch points $c = 0$ and $c = 1$. The variable W is the one in which the small-price behaviour is smooth, and it is the variable the tabulated chart stores.

It is worth recording at the outset what accuracy is even attainable. Writing $\nu(h, v) = \partial C/\partial v$ for the normalized vega, a relative perturbation ε of the input price moves the exact root by the relative amount $\kappa \varepsilon$ with $\kappa = c/(v \nu)$, the relative condition number of the inversion. From (2), deep in the wing $\log c \approx -W$ so $\kappa \approx 1/(2W)$, which is *small*. The wing is benignly conditioned in relative terms, and there is no obstruction to recovering σ to machine precision from prices as small as the smallest positive doubles. Near the upper endpoint the price itself is the wrong variable, since $\kappa \rightarrow \infty$ as $c \rightarrow 1$, but the *complement* $1 - c$ is again benignly conditioned, with a relative perturbation of $1 - c$ moving the root by only $\mathcal{O}(v^{-2})$ times as much. The design consequence, which recurs throughout the paper, is that full accuracy is attainable everywhere provided every formula works with whichever of c and $1 - c$ is small. Since the public entry takes only c , one point deserves precision. For $c \geq \frac{1}{2}$ the subtraction $1 - c$ is *exact* by Sterbenz's lemma [24], so forming the complement loses nothing beyond the loss already committed when the caller rounded the price. The design rule is that no subsequent formula amplifies that rounded complement through an ill-conditioned intermediate. The reference solver of Jäckel [14] makes the same conditioning observation for its own normalization. The residual differences reported in Section 7 arise where the two implementations' internal variables condition differently. One bound, however, no algorithm can cross. A caller who passes the price as a plain double has already rounded $1 - c$, and once $v \gtrsim 8$ that rounding alone moves the root by more than

10^{-13} , growing to 10^{-3} near $v \approx 16.6$ where $1 - c$ reaches machine epsilon. This is a property of the input representation, shared by every double-precision inverter including the reference. The implementation surfaces it explicitly through a status flag rather than refusing the input (Section 6). Quantitatively, on the validation grid of Section 7 ($h \leq 16$, $v \leq 8$) the relative condition number ranges from 7×10^{-4} deep in the wing to 925 at $v = 8$ near the money, so the rounding of the input alone moves the root by between 7×10^{-20} and 1.0×10^{-13} relative. The propagated floor exceeds 10^{-15} only above $v \approx 4.8$ near the money and $v \approx 6$ at $h = 3$. On the box a traded book occupies ($h \leq 3.4$, $v \leq 1.5$) the condition number never exceeds 1.21 and the floor is at most 1.3×10^{-16} , below one rounding of the output. The errors reported in Section 7 are the inverter’s own, scored on the exact double it was given, so the total error of a computed volatility is the propagated input rounding plus the few units in the last place reported there, which is the best any inverter of a rounded price can do.

We score accuracy as the relative error in σ against an independent arbitrary-precision oracle, never against a round trip through a forward pricer. The forward map (1) loses precision to cancellation near $c \rightarrow 0$ and $c \rightarrow 1$, so re-pricing a computed σ and comparing conflates forward-pricer error with inverter error and can inflate the apparent error of *any* correct inverter by two orders of magnitude in the wings. The oracle instead inverts the *given* double-precision price c at 40 significant digits.

Throughout the paper, *machine precision* means a relative error in σ below 10^{-15} (at most a few units in the last place of a double) as *observed on the stated validation sets*. The worst case recorded anywhere in Section 7 is 9.2×10^{-16} for the routed charts alone, six ULP, and 5.7×10^{-16} , four ULP, with the book kernel of Section 5 in front. The claim applies on an explicit domain $\mathcal{D}$: feasible (h, c) with $v = \sigma\sqrt{T} \leq 8$ and $h \leq 16.2$, extended by the rescue band of Section 4.3 ($h \leq 16.5$, $W \lesssim 40$). Outside $\mathcal{D}$ the contract is graded, not silent. The deeper wing corner returns a flagged non-result, and $v > 8$ returns a result flagged `near_saturation`, the accuracy floor being the double representation of $1 - c$ itself. We claim neither correct rounding nor a certified supremum over the continuum. The validation grids are dense and adversarially concentrated at seams and corners, but a polynomial approximation can in principle peak between sampled points, and a proven bound would require interval certification of every cell and kernel, which we leave as future work.

4 The tabulated chart and endpoint charts

The feasible domain is tiled by four approximants that share no data-dependent iteration. A bivariate table covers the interior, and three analytically matched charts cover the endpoints, each most accurate exactly where the others fail. Together they cover the whole feasible domain with no additional two-dimensional tables, and Section 5 then shows that in the inverse’s own coordinates one of them, made exact, reaches across the others over the whole traded book. This mirrors the structure of the base integral (2), whose boundary layer at $t = 0$ is resolved by a matched coordinate, whose regular Gaussian tail is resolved by an elementary price transform, and whose middle, where neither simplification applies, is the table’s natural home.

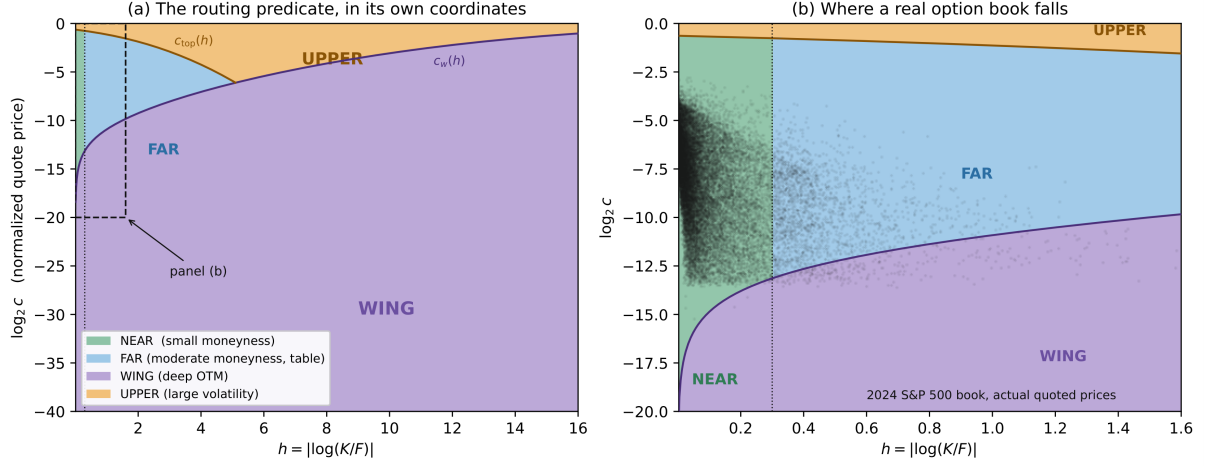


Figure 2: Routing of the input domain, drawn in the coordinates the classifier actually uses. **(a)** The strip $0 < c < 1$ of normalized quote prices against moneyness $h = |\log(K/F)|$, partitioned into the four charts. No inversion is involved. Each seam is a *threshold price* returned by a frozen one-dimensional polynomial in h — $c_w(h) = C(h, h/\sqrt{7.6})$ below, the iso- W contour $W = 3.8$, and $c_{\text{top}}(h) = C(h, 1.85)$ above, a single ceiling shared by both tabulated charts — and a quote is classified by comparing its own price against them, then by integer tests on its IEEE-754 bit fields. The two curves are therefore the routing predicate itself rather than a picture of its consequences. Because $C(h, \cdot)$ maps $(0, \infty)$ onto $(0, 1)$ bijectively, every point of the strip is a genuine quote, and there is no infeasible region. The tabulated chart closes where the two seams meet, so its extent is fixed by the crossing of two validated boundaries rather than by any chosen cut-off. The ceiling’s height is set the same way. Each chart hands over where its neighbour is validated, not where symmetry would suggest. The large-volatility chart receiving these quotes is machine-precise from $v = 1.55$ upward, while the small-moneyness chart’s matched expansion begins to extrapolate as $v \rightarrow 2$, and 1.85 is the highest level at which both tabulated charts remain validated, so one seam serves both. **(b)** A zoom (boxed in (a)) with an entire year of tradeable 2024 S&P 500 end-of-day quotes overlaid as their *actual quoted prices*. About 92% of this book falls in the small-moneyness chart and a further 7% in the tabulated chart, while the large-volatility region carries essentially none and the deep wing retains only the quotes that genuinely require it.

4.1 The small-moneyness chart: NEAR

For h below the threshold $h < 0.3$ the geometry collapses toward the explicit at-the-money line $c = \text{erf}(\sigma/2\sqrt{2})$. The level sets of c crowd against an exactly known curve, and direct two-dimensional approximation of $w(h, c)$ degenerates precisely where the answer is simplest. The cure is a matched coordinate that absorbs the collapse. Scale the price by the leading h -dependence,

$$\rho = \frac{c}{\text{expm1}(h)}, \quad (4)$$

and let $A = B^{-1}(\rho)$ where

$$B(A) = \frac{\varphi(A)}{A} - \Phi(-A) = \int_A^\infty \varphi(u) u^{-2} du \quad (5)$$

is an h -free, strictly decreasing universal function arising from the $t \rightarrow 0$ layer of (2). The substitution $t = h/u$ maps the layer factor $e^{-h^2/(2t^2)}$ to the Gaussian $\varphi(u)$ and produces exactly the integral B , so the coordinate is dictated by the integral representation rather than chosen. The matched variable is $s = h/A$, which reduces to v as $h \rightarrow 0$, and in it the volatility has an

even, regular expansion

$$v(h, c) = v_0(\kappa s) + h^2 V_2(s) + h^4 V_4(s) + h^6 R(s, h), \quad (6)$$

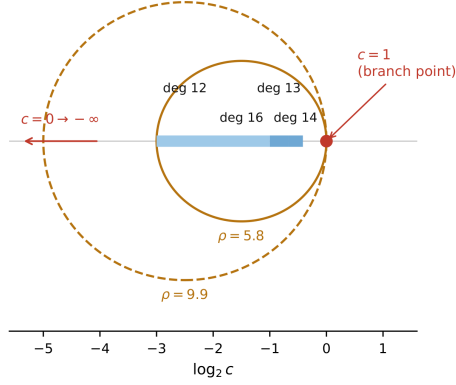
where $v_0(\cdot) = 2\sqrt{2} \operatorname{erf}^{-1}(\cdot)$ is the exact at-the-money kernel (the pair (h, c) identifies the total standard deviation v , not the annualized σ , which only the convenience wrapper recovers as $\sigma = v/\sqrt{T}$), $\kappa = 1/\sqrt{2\pi}$, and V_2, V_4 are elementary functions of s alone. Evenness in h is not an accident of fitting but the put–call symmetry $h \mapsto -h$ of the underlying density, and it is what makes the expansion efficient—each retained order gains a uniform factor h^2 . This was verified directly, by halving h at fixed v and observing the residual error of the one-, two- and three-term truncations divide by 4, 16 and 64. The leading term alone is uniformly second-order accurate through the deep-OTM corner (small c jointly with small h) that defeats naive tabulation. The implementation choices that complete the chart, the frozen residual R and the Chebyshev replacements of V_2, V_4 and B^{-1} , are recorded in Appendix A.

4.2 The tabulated chart: FAR

Over the interior of the domain the inversion is resolved by direct evaluation of a precomputed table, with no iteration at all. The rationale is a cost observation before it is an accuracy one. A machine-precision iterative inverter spends essentially all of its time in its accurate forward evaluations. A seed plus two Householder corrections costs two evaluations of a carefully written Black function, each dominated by error-function calls. A table evaluation replaces all of that by one polynomial sweep whose operation count is fixed, known at compile time, and free of data-dependent branches, which is also exactly the shape of computation that vectorizes. The questions that decide whether the approach reaches machine precision are which variable to tabulate, in which coordinate, and at what polynomial degree. All three answers come from the analytic structure of the inverse map.

A prior question is whether tabulation is needed at all, or whether some change of variable turns the interior inversion into a classical quantile that could simply be evaluated. It cannot. Read as a quantile identity, the Black–Scholes variance law meets the generalized inverse Gaussian family only at the orders $p = \pm \frac{1}{2}$, realized on the variance and on the inverse Gaussian side respectively, and the only monotone re-coordinatizations that preserve that family’s exponent are scaling, which moves the tilt parameters, and reciprocation, which flips the sign of the order [2, 15, 27]. Neither removes the inversion. Scaling leaves it in place, and reciprocation exchanges the two orders, which is the $c \leftrightarrow 1 - c$ pairing the two endpoint charts already exploit. Representations outside this elementary orbit require a genuine change of variable, carrying special functions of its own. Tabulating the interior is therefore forced rather than merely convenient, and the three questions above are the right ones to ask of it.

The approximation target is the transformed variable W of (3) rather than σ or w themselves. W is the exponential scale variable of the wing asymptotics, it is nearly linear in $\log c$ across each cell, and fitting σ directly was measured to cost one to two additional Chebyshev degrees per cell for no benefit. The abscissa is $\lambda = \log c$. The choice of λ rests on a design principle about where the inverse map ceases to be analytic, which we state and then say precisely how far we have verified it. Viewed as a function of the price c at fixed h , the inverse $w(h, \cdot)$ has branch points at the two endpoints $c = 0$ and $c = 1$, and the distances from an interior price c to these branch points are exactly c and $1 - c$, *independently of h* . The convergence of a Chebyshev approximation on a cell is governed by the Bernstein ellipse through the nearest *complex* singularity [21, 30]. At the time the table was built we had not proved that no complex singularity lies nearer than the two real endpoints, so we treated the endpoint distances as a predictive design principle and tested it against the fitted degrees of all 181 cells, where a nearer singularity would announce itself as degrees systematically above the prediction. That principle is now partly a theorem, as recorded at the end of this subsection. The observed

(a) cells and their Bernstein ellipses on the $\log_2 c$ axis

(b) fitted degree vs. per-cell prediction

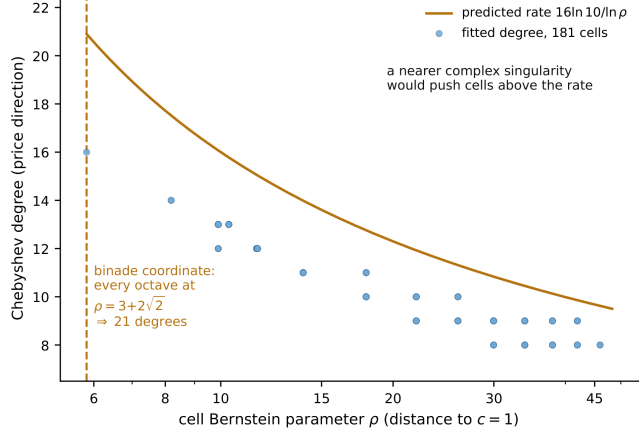


Figure 3: The branch-point geometry that sizes the tabulated chart. **(a)** The cells nearest the money in one h -band of the shipped table, on the $\log_2 c$ axis, with the Bernstein ellipses of two of them drawn in place. Each cell’s largest singularity-free ellipse passes exactly through the $c = 1$ branch point, so the Bernstein parameter ρ grows as cells recede from the money, while $c = 0$ recedes to $-\infty$ in this coordinate. Without sub-splitting, the octave adjacent to the money would degenerate toward $\rho = 1$. Sub-splitting those octaves (dark cells) together with the seam to the large-volatility chart keeps the worst normalized distance at $x = 3$, the same worst case as the binade coordinate, so $\rho \geq 3 + 2\sqrt{2}$ for every cell. **(b)** The fitted price-direction degrees of all 181 cells against each cell’s ρ , reconstructed from the shipped routing tables, with the predicted rate $16 \ln 10 / \ln \rho$ overlaid. In the binade coordinate every octave sits at $\rho = 3 + 2\sqrt{2}$ (dashed vertical), a flat floor of about twenty-one degrees. The fitted degrees track the predicted rate from below at a roughly uniform offset, the unknown order-one prefactor of the asymptotic rate. The endpoint distances remain a predictive design principle, not a proof that no complex singularity lies nearer. A nearer one would push cells systematically above the rate, and none appears.

degrees match the prediction throughout (Figure 3). In the raw mantissa (binade) coordinate the $c = 0$ branch point sits one octave to the left of every octave, in a self-similar way, which pins the Bernstein parameter at $\rho = 3 + 2\sqrt{2}$ and predicts a floor of roughly twenty-one degrees per octave everywhere (confirmed by fits), so a transcendental-free table in that coordinate is a dead end, elegant as it looks. A conformal coordinate adapted to the annulus fares even worse in practice (fitted degrees near one hundred), because it compresses the real interval against the boundary. In $\lambda = \log c$ the distance to the branch point grows with the logarithmic cell width, and the fitted degrees fall to a median of ten, never exceeding sixteen. These are measured degrees from the table generator, not estimates.

On each cell the table stores a bivariate Chebyshev surrogate

$$W \approx \sum_{i=0}^{d_h} \sum_{j=0}^{d_\lambda} c_{ij} T_i(x_h) T_j(x_\lambda), \quad (7)$$

with $x_h, x_\lambda \in [-1, 1]$ affine images of the local h -band and $\log c$ interval. The degrees, the cell geometry in numbers, the bit-field routing to a cell, the extent of the bands and the oracle-gated generation are engineering rather than mathematics and are collected in Appendix A. The logarithmic prefactor that explains the offset in Figure 3(b) is derived there as well.

4.3 The deep wing: WING

For prices below the table’s reach the inversion works on the logarithm of the endpoint factorization of Section 3, with the dominant asymptotic behaviour peeled off analytically. This is the extreme-strike regime whose leading-order asymptotics are classical (the moment formula of Lee [18]), but the chart must recover the map itself to full accuracy, not its leading order. Taking logarithms of $c = \gamma(h)e^{-W}W^{-3/2}H(W)$ turns the inversion into the implicit equation

$$W = \tilde{L} - \frac{3}{2} \log W + \log H(W), \quad (8)$$

where $\tilde{L}$ collects the price and h data. Because the equation lives on the logarithm of the price, its conditioning is the benign $1/(2W)$ of Section 3, and the inverter is validated on prices down to 10^{-320} , that is, across the entire subnormal range of double precision. The shared logarithm kernel of Section 6 is extended by an exact power-of-two rescaling so that subnormal prices lose no accuracy on entry.

What makes the chart vectorizable is the evaluation of $\log H$. A runtime quadrature, the natural first implementation, has neither a fixed operation count in the useful sense (forty exponentials per evaluation) nor competitive speed, so it is retired to the offline oracle role and replaced by a frozen two-piece bivariate Chebyshev fit. The fitted quantity is deliberately not $\log H$ itself but the peeled, weighted residual

$$S(u, \beta) = \frac{\log H}{u} + \left(\beta + \frac{3}{2}\right), \quad u = \frac{1}{W}. \quad (9)$$

Peeling the leading asymptotic term $-(\beta + 3/2)/W$ removes the dominant variation, and dividing by u makes the fit’s equioscillating error automatically proportional to the σ -tolerance, which grows like W . Fitting the raw residual instead was measured to demand degrees beyond thirty-six where the weighted form needs twenty-six. Two pieces with a generous overlap cover $W \in [2.5, 260]$ (668 coefficients, 5.2 KB), the eight-term asymptotic series takes over beyond, and the piece-or-series regime is selected *once* per quote from $\tilde{L}$, so no per-step branching enters the kernel. The derivative $\partial_W \log H$ that drives the Newton steps is computed from the *same* coefficients by the Chebyshev derivative recurrence, so the value and slope seen by the iteration are consistent by construction. The solve is a fixed count of three Newton steps from a tabulated seed. Started from the cheap logarithmic seed $W_0 = \max(\tilde{L} - \frac{3}{2} \log \tilde{L}, 2.6)$ alone, the iteration needs six steps, and that count is delicate. Five passed a sparse de-risk grid, yet a dense sweep exposed them as one step short at the high- β , low- W corner, where convergence is slowest (Appendix C records the incident). Seeding instead with W_0 plus a small peeled-residual correction, one bivariate Chebyshev surface of degree (8, 8) in $(\tilde{L}, \beta)$ per piece (81 coefficients each, 1.3 KB in total), starts the iteration inside the quadratic basin and cuts the fixed count from six to three, halving the chart’s dominant cost. A direct table for W itself, dispensing with Newton entirely, fails on degree, because the small- W edge is genuinely asymptotic, but as a *seed* the low-degree surface is exactly enough. The reduction is gated the way the step-count incident taught. Against the oracle over 6,325 points spanning the full service domain, including the slowest-converging corner and the series regime, the three seeded steps reach a worst volatility error of 8.0×10^{-16} , marginally better than the six bare-seeded steps (8.4×10^{-16}), with no point above 10^{-15} . The chart shares the fixed-operation structure of the other charts, so the batch drivers evaluate it eight lanes at a time like the rest.

One boundary of the fitted table must be handled rather than clamped. The fit covers $\beta = h^2/16 \leq 16.4$, i.e. $h \leq 16.2$, corresponding to strikes more than 10^7 times the forward. Benchmark grids nevertheless reach past it, and a clamped evaluation there would return a plausible-looking volatility wrong in the third digit, the worst kind of failure. Quotes beyond the fitted β therefore take a rare scalar branch. A seed obtained by dropping the second Black term and solving $\Phi(v/2 - h/v) = c$ in closed form (then restoring the dropped term by three fixed-point

passes, worst seed error 0.23% over $h \in [16.2, 30]$), finished by the same three Householder steps as the warm-start entry. The branch is validated for $W \lesssim 40$. The still-deeper corner, unreachable by any market data, returns a quiet NaN with an explicit out-of-domain status rather than a number (Section 6). Both the scalar router and the batch drivers call the identical branch function, so the bit-identity invariant is preserved structurally.

4.4 The large-volatility chart: UPPER

At the upper price endpoint, $v > 1.85$ and $c \rightarrow 1$, the complement of (2) is

$$1 - c = k e^{h/2} \int_v^\infty e^{-t^2/8} e^{-h^2/(2t^2)} dt, \quad (10)$$

and the structural point that makes this endpoint fundamentally easier than the left one is that the layer factor is *regular* there, with $e^{-h^2/(2t^2)} \rightarrow 1$ as $t \rightarrow \infty$, with an expansion in h^2/t^2 that converges absolutely for every $t > 0$. There is no layer to match, and termwise integration (justified by dominated convergence) yields an exact, absolutely convergent series rather than an asymptotic one. Introducing the ladder $B_n(A) = \int_A^\infty \varphi(u) u^{-2n} du$, which satisfies the elementary recursion $B_{n+1}(A) = [\varphi(A)A^{-(2n+1)} - B_n(A)]/(2n+1)$ with $B_0(A) = \Phi(-A)$ and $B_1 = B$, one obtains with $x = v/2$

$$\frac{(1-c)e^{-h/2}}{2} = \Phi(-x) + \sum_{n \geq 1} \frac{(-h^2/8)^n}{n!} B_n(x). \quad (11)$$

Every identity in this subsection was checked against 30-digit arithmetic across the strip, with exact agreement to working precision.

Two consequences follow immediately. Setting $h = 0$ in (11) identifies the leading inverse as the *same* at-the-money kernel as on the left, evaluated at the transformed price

$$\tilde{c} = 1 - (1-c)e^{-h/2}, \quad v \approx 2\sqrt{2} \operatorname{erf}^{-1}(\tilde{c}) + h^2 V_2^R + h^4 V_4^R + \dots, \quad (12)$$

where $V_2^R = -B_1(x_0)/(4\varphi(x_0))$ and the higher coefficients are, through the ladder recursion, polynomials in $1/x_0$ and the Mills ratio $m(x) = \Phi(-x)/\varphi(x)$. The Mills ratio is the only genuinely new one-dimensional function the endpoint requires. And since $B'_n(x) = -\varphi(x)x^{-2n}$, the series for the derivative of the right-hand side of (11) resums in closed form,

$$g'(x) = -\varphi(x) e^{-h^2/(8x^2)}. \quad (13)$$

The truncated ladder alone is excellent deep in the endpoint, where the expansion parameter $W = h^2/(2v^2)$ is small, but degrades toward moderate v . Table 1 reports the verified relative error of the three-term truncation across the strip. The chart is therefore finished by iterating on (11) itself. The equation is exact and the derivative (13) is cancellation-free and free of error functions. Moreover, because $\log|g'|$ is the elementary expression $\operatorname{const} - x^2/2 - h^2/(8x^2)$, the higher derivative ratios needed by a third-order Householder step are closed-form rationals, $g''/g' = -x + h^2/(4x^3)$ and $g'''/g' = (g''/g')^2 - 1 - 3h^2/(4x^4)$, so an order-four step costs the same one price evaluation as a Newton step. This is the same economics that motivates Householder refinement in Jäckel [14], and it halves the finisher. The implementation performs a *fixed* count of three Householder steps with no early exit. The count was sized by measuring pure iteration convergence against exact prices. The seed degrades to a relative error of 0.27 at the worst corner of the strip (near the wing seam, $W \approx 3$), from which the fourth-order steps contract through 10^{-2} , 10^{-7} and 10^{-27} , so the third step lands with eleven orders of margin while two steps demonstrably do not suffice. A fixed count costs a little at benign points but buys determinism and lane-uniformity in the vectorized path. Prices above one half are handled throughout in the complement form. The subtraction $1 - c$ is exact by Sterbenz's lemma, and the point of the complement form is that no subsequent formula amplifies that rounded complement. This, with the cancellation-free transform $c \mapsto \tilde{c}$, is why the chart retains full precision as $c \rightarrow 1$ where a $\sqrt{FK}$ -normalized formulation measurably does not (Section 7).

h	v	$W = h^2/(2v^2)$	rel. error of $V_0^R + h^2V_2^R + h^4V_4^R$
0.3	7.0	0.0009	1.8×10^{-12}
1.0	7.0	0.0102	2.4×10^{-9}
2.0	7.0	0.0408	1.5×10^{-7}
6.6	7.0	0.44	1.7×10^{-4}
2.0	2.5	0.32	2.0×10^{-3}

Table 1: Verified relative accuracy of the three-term ladder seed of the large-volatility chart, before the Householder finisher. The expansion parameter is $W = h^2/(2v^2)$. The finisher’s exact equation and closed-form derivatives bring every point to machine precision.

4.5 One family, analytically bounded, and derived seams

A single identity ties the charts together. With

$$H(W; a, \beta) = \int_0^\infty e^{-r} \left(1 + \frac{r}{W}\right)^{-a} e^{-\beta/(W+r)} dr, \quad (14)$$

the substitution $u^2 = A^2 + 2r$ gives, exactly,

$$B_n(A) = \varphi(A) A^{-(2n+1)} H\left(\frac{A^2}{2}; n + \frac{1}{2}, 0\right), \quad (15)$$

and the same family evaluated at $(a, \beta) = (3/2, h^2/16)$ and $(1/2, h^2/16)$ furnishes the two deep wings. Every special function appearing anywhere in the architecture is therefore one two-parameter family, and only two genuinely one-dimensional, h -independent inverses (erf^{-1} and B^{-1}) ever appear. In matched coordinates, all the h -dependence is a smooth prefactor times a regular even perturbation. For $\beta = 0$ the family is Stieltjes,

$$H(W; a, 0) = \frac{1}{\Gamma(a)} \int_0^\infty \frac{t^{a-1} e^{-t}}{1 + t/W} dt, \quad (16)$$

so its continued-fraction convergents bound it from alternating sides [1], and B , every B_n , and by monotone inversion B^{-1} admit rigorous two-sided enclosures at any order (the one place the word *certified* applies in its strict sense), and these enclosures supply the truth values against which the endpoint kernels were gated. For $\beta > 0$ the Stieltjes property fails, and the mechanism is an identity rather than a numerical observation. The same Stieltjes kernel represents the family with a *signed* density,

$$H\left(W; \frac{3}{2}, \beta\right) = \frac{1}{\Gamma(3/2)} \int_0^\infty t^{1/2} e^{-t} \frac{\sin(2\sqrt{Xt})}{2\sqrt{Xt}} \frac{dt}{1 + t/W}, \quad X = \frac{\beta}{W} = \frac{w}{8}, \quad (17)$$

because the inverse Laplace transform of $z^{-3/2} e^{-X/z}$ is $\sin(2\sqrt{Xt})/\sqrt{\pi X}$. The oscillating factor is the sign change, and it degenerates to unity at $\beta = 0$. One consequence closes the circle with the forward evaluation discussed at the end of this section. Writing $a = -d_1/\sqrt{2}$ and $b = -d_2/\sqrt{2}$, the two arguments the shared price kernel already forms, and using the exact relation $b^2 = a^2 + h$,

$$H\left(W; \frac{3}{2}, \frac{h^2}{16}\right) = \sqrt{\pi} W e^{-X} \frac{\text{erfcx}(a) - \text{erfcx}(b)}{b - a}, \quad (18)$$

so the wing member of the family is a divided difference of the very erfcx the refiners evaluate. Both identities were checked against adaptive quadrature to forty digits. The exact identity $H(W; a, \beta) = H(W; a, 0) - \int_0^\infty \sqrt{\beta/\tau} J_1(2\sqrt{\beta\tau}) e^{-W\tau} H(W(1+\tau); a, 0)/(1+\tau) d\tau$, obtained by writing $1 - e^{-\beta s}$ as the classical J_1 Laplace transform [25, Ch. 10] and applying it under the

integral sign, expresses $H(\cdot; a, \beta)$ through the $\beta = 0$ family (we exercise it numerically to 10^{-21} in the generation pipeline), so two-sided enclosures extend to the wing evaluators as well.

The seams between the charts are derived, not tuned, and they are more than that, since each is a level set of the exact analyticity geometry of the inverse. Fix the normalized price $\rho = c/(e^k - 1)$ rather than the raw price. The physical branch $v_\rho(k)$ is then odd in k , vanishes at $k = 0$, and is holomorphic in $|k| < \min\{a_\rho, 2\pi\}$ with

$$a_\rho = \log\left(1 + \frac{1}{\rho}\right), \quad (19)$$

this being the value of $|k|$ at which the moving target $\rho(e^k - 1)$ collides with the asymptotic value 1, while the cap at 2π is the first nonzero solution of $e^k = 1$. The natural coordinates of the inverse are therefore $\theta = k/a_\rho$, the position of a quote as a fraction of its own analyticity radius, together with a_ρ itself, and in those coordinates both shipped seams are coordinate lines. The ceiling is the curve $\theta = \theta^*$ with $\theta^* = \text{erf}(1.85/(2\sqrt{2})) = 0.645034092553535$ exactly, this being the $k \rightarrow 0$ limit of the $v = 1.85$ contour, and freezing θ^* across the whole small-moneyness band moves the seam only within $v \in [1.85000, 1.85178]$. The wing seam is the line $\rho = B(\sqrt{7.6})$, which is simultaneously a level set of a_ρ and of $A = B^{-1}(\rho)$ because all three are functions of ρ alone, and freezing it moves the seam only within $W \in [3.79625, 3.8]$. Both ranges are extrema located by optimization at fifty digits rather than grid samples. That the seams are this flat in the intrinsic coordinates while the corresponding prices sweep eight decades is precisely why their price-space description needs polynomials at all. Independently of the geometry, the truncation error envelopes of the two endpoint expansions follow the scaling laws (asymptotically motivated, confirmed by high-precision measurement of the envelopes) $E_{\text{near}} \propto v^2 h^{2N+2}$ and $E_{\text{upper}} \propto (h^2/2v^3)^{N+1}$, and equating envelopes across the wide overlap bands, where two charts are simultaneously accurate, yields thresholds that are constants in the matched coordinates, in agreement with the geometric placement. The implementation freezes both as one-dimensional price curves, each a single Chebyshev polynomial in h : the wing seam $c_w(h) = C(h, h/\sqrt{7.6})$ (the level set $W = 3.8$) and the ceiling $c_{\text{top}}(h) = C(h, 1.85)$, one curve shared by both tabulated charts. The ceiling sits at 1.85 rather than 2 so that the exact-equation Householder refinement of Section 4.4, rather than an extrapolated finisher, owns the corner where the charts meet. The large-volatility chart is machine-precise from $v = 1.55$ upward, whereas the small-moneyness chart's matched expansion begins to extrapolate as $v \rightarrow 2$, and 1.85 is the highest level at which both tabulated charts remain validated. Freezing the seams as polynomials matters beyond tidiness. Routing then costs two short polynomial sweeps rather than any runtime price evaluation, which is what allows the classification itself to vectorize (Section 6). A fitted seam differs from the exact boundary only within its fit error ($\sim 10^{-16}$ relative), reclassifying at most quotes within a ulp of the seam—inside the overlap bands, where both neighbouring charts are machine-precise, so nothing delicate happens at a crossing. The dense seam sampling reported in Section 7 confirms it.

The resulting coverage map is this. The deep wing owns $W \gtrsim 3.8$, including the feasibility pinch at very large h , which is a wing region in disguise since the feasible price there is necessarily small. The small-moneyness chart owns $h < 0.3$ up to the $v = 1.85$ ceiling. The large-volatility chart owns everything above its seam, including the whole strip $h > H_{\text{box}}$ that is not wing. The tabulated chart owns the remainder. The four regimes tile the domain exactly, and on a broad random feed no feasible quote fails to route.

The accurate forward. As in Jäckel [14], the precision floor of the whole method is the accuracy of the forward evaluation used inside the places that refine, the large-volatility Householder finisher and the warm-start entry of Section 6. We evaluate the normal functions there through a scaled complementary error function erfcx built on frozen Chebyshev pieces in the spirit of Cody [5], three degree-22 pieces covering $[0, 7.2]$, selected per lane by blends off the

critical path, which halves the sequential Clenshaw chain relative to a single degree-36 fit and extends the domain to the arguments the warm-start basin requires, and an exponential reduced by a Cody–Waite splitting of $\log 2$ followed by a Chebyshev polynomial on $[-\frac{1}{2} \log 2, \frac{1}{2} \log 2]$ and an exact power-of-two scaling. Both Black terms of a price residual are produced by one shared kernel that evaluates the $\pm$ pair from a single erfex core and one shared normal tail, with a compensated square killing the rounding of the squared argument, so each refinement step costs two erfex and four exponential evaluations rather than the naive six. The refiners therefore perform no library erf, erfc or exp call, their rounding behaviour is fully determined by the frozen coefficients, and the rational quantile seed’s 10^{-9} error is immaterial because the equation being solved is exact.

5 The intrinsic coordinates and the book kernel

In the inverse’s intrinsic coordinates the small-moneyness chart is exact, every coefficient beyond one tabulated function of a single variable being a rational number. The matched expansion (6) fits three functions, V_2 , V_4 and R , as oracle-gated Chebyshev surrogates. The form below replaces all three by exact rational rows. Fix the normalized price $\rho = c/(e^h - 1)$ as in Section 4.5 and use the intrinsic coordinates

$$a = \log\left(1 + \frac{1}{\rho}\right), \quad \theta = \frac{h}{a}, \quad \tau = \theta^2, \quad V(\tau, a) = \frac{v}{\theta}, \quad (20)$$

so that a is the exact analyticity radius of the fixed- ρ branch in h (capped at 2π), θ is a quote’s position as a fraction of that radius, and V is even in θ by the oddness of the branch. Two facts fix the chart. The first is the radius statement of Section 4.5, proved in Schadner [28]. At fixed real a the series of V in τ has radius exactly 1 for $a < 2\pi$ and $(2\pi/a)^2$ above, because the only obstructions are the collisions of the moving target $\rho(e^k - 1)$ with the asymptotic values of the direct map, at $k = \pm a + 2\pi i n$ and $k = 2\pi i n$, and the direct map has no finite critical points, its v -derivative being the Gaussian $\varphi(d_1)$. The second is that the τ -rows are exact. Write $A_0(a) = B^{-1}(\rho)$ for the $h \rightarrow 0$ inverse of Section 4.1, $V_0 = a/A_0$, and expand $V = \sum_m V_m(a)\tau^m$. Then

$$V_m(a) = V_0(a)^{2m+1} P_m(A_0(a)^2), \quad P_0 = 1, \quad P_1 = \frac{1}{24}, \quad P_2 = \frac{21-2y}{5760}, \quad \dots \quad (21)$$

with every P_m a polynomial of degree at most $m - 1$ with rational coefficients ($P_3 = (16y^2 - 174y + 1143)/2903040$, and so on, and through order 22 the degree is exactly $m - 1$). The structure is a theorem rather than an observation, and its proof is the recursion the generator runs.

Proposition 1. *Fix $\rho \in (0, \infty)$, let $A_0 = B^{-1}(\rho)$ and $V_0 = a/A_0$. The branch of the price equation with $V(0, a) = V_0$ has the formal expansion $V(\tau, a) = \sum_{m \geq 0} V_m(a)\tau^m$ with $V_m = V_0^{2m+1} P_m(A_0^2)$, where $P_0 = 1$ and, for every $m \geq 1$, P_m is a polynomial with rational coefficients of degree at most $m - 1$.*

Proof. Write k for the log-moneyness h of (1), $v = k/A$, and $q = k^2 = a^2\tau$.

Step 1, the price equation in closed form. By (1), $c = \Phi(v/2 - A) - e^k \Phi(-v/2 - A)$, and since $\varphi(A \mp s) = \varphi(A) e^{\pm As - s^2/2}$, $\Phi(v/2 - A) = \Phi(-A) + \varphi(A) I_+$ and $\Phi(-v/2 - A) = \Phi(-A) - \varphi(A) I_-$ with $I_{\pm} = \int_0^{v/2} e^{\pm As - s^2/2} ds$. Hence $\rho = c/(e^k - 1) = -\Phi(-A) + \varphi(A) (I_+ + e^k I_-)/(e^k - 1)$. Expanding $e^{-s^2/2}$ and substituting $s = t/A$, $I_{\pm} = \sum_{n \geq 0} \frac{(-1)^n}{2^n n!} A^{-2n-1} \int_0^{k/2} t^{2n} e^{\pm t} dt$. The $n = 0$ term of $I_+ + e^k I_-$ is $(e^k - 1)/A$, which together with $-\Phi(-A)$ is exactly $B(A)$; the remaining terms give

$$\rho = B(A) + \varphi(A) \sum_{n \geq 1} \frac{(-1)^n}{2^n n!} A^{-2n-1} G_n(k), \quad G_n(k) = \frac{\int_0^{k/2} t^{2n} \cosh(k/2 - t) dt}{\sinh(k/2)}. \quad (22)$$

Substituting $t = (k/2)(1-u)$ gives $G_n(k) = (k/2)^{2n} \int_0^1 (1-u)^{2n} \cosh(ku/2) du / [\sinh(k/2)/(k/2)]$. The Beta integrals of the cosh series and the Bernoulli series of $x/\sinh x$ are rational, so $G_n(k) = k^{2n} g_n(k^2)$ with $g_n \in \mathbb{Q}[[q]]$ and $g_n(0) = 4^{-n}/(2n+1)$. Collecting powers of q ,

$$\rho = B(A) + \varphi(A) S(A, q), \quad S(A, q) = \sum_{m \geq 1} q^m S_m(A^{-1}), \quad (23)$$

where each S_m is an odd polynomial in A^{-1} with rational coefficients and degree at most $2m+1$.

Step 2, formal inversion. From $B'(A) = -\varphi(A)/A^2$ and $\varphi' = -A\varphi$, $B^{(j)} = \varphi b_j$ with $b_1 = -A^{-2}$ and $b_{j+1} = b'_j - Ab_j$; likewise $(\varphi S_m)^{(j)} = \varphi \sigma_{m,j}$ with $\sigma_{m,0} = S_m$ and $\sigma_{m,j+1} = \sigma'_{m,j} - A\sigma_{m,j}$. All b_j and $\sigma_{m,j}$ are Laurent polynomials in A with rational coefficients, of parity $(-1)^{j+1}$ under $A \mapsto -A$, the powers of A^{-1} lying in $[2-j, j+1]$ for b_j and in $[3-j, 2m+1+j]$ for $\sigma_{m,j}$. Write $A(q) = A_0 + \delta$ with $\delta = \sum_{m \geq 1} c_m q^m$. Expanding (23) about A_0 and dividing by $\varphi(A_0) \neq 0$,

$$\sum_{j \geq 1} \frac{b_j(A_0)}{j!} \delta^j + \sum_{j \geq 0} \sum_{i \geq 1} \frac{\sigma_{i,j}(A_0)}{j!} q^i \delta^j = 0, \quad (24)$$

in which no exponential, error function or density survives. The coefficient of q^m in the $j=1$ term of the first sum is $-c_m/A_0^2$, and every other contribution to q^m involves only $c_1, \dots, c_{m-1}$, so with $[\cdot]_m$ the coefficient of q^m ,

$$c_m = A_0^2 \left(\sum_{j \geq 2} \frac{b_j(A_0)}{j!} [\delta^j]_m + \sum_{j \geq 0} \sum_{i=1}^m \frac{\sigma_{i,j}(A_0)}{j!} [\delta^j]_{m-i} \right), \quad (25)$$

which is the recursion the generator evaluates in exact rational arithmetic. By induction on m , c_m is an odd polynomial in A_0^{-1} with rational coefficients and degree between 1 and $2m-1$. A product of j of the c_i with indices summing to m has parity $(-1)^j$ and degree in $[j, 2m-j]$, so every term inside the brackets is odd with degree in $[2, 2m+1]$, oddness excludes degree 2, and the factor A_0^2 lowers the range by two.

Step 3, the rows. $V = a/A = V_0(1 + \sum_m d_m q^m)^{-1}$ with $d_m = c_m/A_0$, even in A_0^{-1} with degree in $[2, 2m]$; hence $V = V_0 \sum_m e_m q^m$ with $e_0 = 1$ and, for $m \geq 1$, e_m a polynomial in $d_1, \dots, d_m$ of total order m , even with degree in $[2, 2m]$. With $q = a^2 \tau$ and $a = V_0 A_0$, $V_m = V_0 a^{2m} e_m = V_0^{2m+1} A_0^{2m} e_m(A_0)$, and $A_0^{2m} e_m$ is a polynomial in A_0^2 of degree at most $m-1$ with rational coefficients. $\square$

The first step of (25) returns $c_1 = -1/(24A_0)$ and $P_1 = 1/24$. The rows to order 22 that the kernel stores are (25) carried out exactly, checked term by term against the 40-digit solve, and their leading coefficients carry the Bernoulli numbers of the $x/\sinh x$ series of Step 1. Whether the P_m obey a closed recurrence in m alone, which would generate them at runtime, is open, so the kernel stores them.

The consequence of Proposition 1 for the runtime is that the chart needs one tabulated function and nothing else. With $x = h^2/A_0^2$ and $y = A_0^2$,

$$v = \frac{h}{A_0} S(x, y), \quad S(x, y) = \sum_{m=0}^M x^m P_m(y), \quad (26)$$

and the only fitted object is $G(a) = 1/V_0(a) = A_0(a)/a$, a smooth function of one variable with $G(0) = 1/\sqrt{2\pi}$, whose Chebyshev series on the whole band $a \in [0, a_{\text{wing}}]$ decays at the rate set by the poles of V_0 at $a = \pm 2\pi i$ (Bernstein parameter 3.84), so that 25 coefficients reach 10^{-17} . On the book's range $\tau \leq 0.035$ ten rows of (26) truncate below 2.5×10^{-18} , and the assembled chart, table and rows together, agrees with the 40-digit solve to 1.9×10^{-17} relative. Per quote the evaluation is one 25-term Clenshaw sweep, 55 fused multiply-adds of exact rational constants, and three divisions (E/c , one inside the logarithm, h/A_0), against the matched chart's seven

divisions and about 130 fitted coefficients across five polynomials. The error budget reduces to a one-dimensional Bernstein statement for G plus the explicit first dropped term of the series. On a 40-digit truth set of 842 points in the band the chart’s worst relative error is 5.0×10^{-16} (2.25 ULP), and the scalar, AVX2 and AVX-512 builds return byte-identical output vectors on the 344,178 small-moneyness quotes of the coverage population, as does the CUDA transcription compiled for the host. Above $\tau = 0.035$ the series slows toward its $\tau = 1$ singularity and the matched chart of Section 4.1 answers instead. Section 7 times both.

One kernel for the book. Nothing in (21) knows about the small-moneyness band. Read in h , the radius statement says the series converges for $|h| < \min(a, 2\pi)$, which in the wing is never less than 2π , so the same rows reach across the interior into the deep wing. Two things are needed to make that a kernel. The first is a table that runs to large a , where $V_0 \sim \sqrt{2a}$, with one cell for G in a on $[0, 2\pi]$ and one for the flat function $V_0/\sqrt{a}$ in the variable $u = 1/\sqrt{a}$ on $[2\pi, 700]$, which covers prices down to 10^{-300} , at a cost of 23 and 40 coefficients. Tabulating $1/V_0$ itself on the second cell, where it falls to 0.05, would let the Clenshaw sum’s absolute rounding become a relative error of several units in the last place, and $u V_0/\sqrt{a}$ is $1/A_0$ directly, so the wing side needs no division for h/A_0 . The second is that above $a = 2\pi$ the nearest obstruction is the periodic return $h = \pm 2\pi i$ rather than the real pair, so the raw series in h^2 converges slowly toward the wing. Re-expanding in the conformal variable $q = z/(\sqrt{1+z} + 1)^2$, $z = h^2/4\pi^2$, which maps that cut to the unit circle while real h stays at small q , restores a fast rate. In q the series is again a triangular polynomial with fixed constants, combinations of the P_m with binomials and powers of $4\pi^2$, evaluated by the same Horner sweep in the variable $w = 1/A_0^2$. Twelve rows reach $h = 3$ and sixteen reach $h = 4$ at any a above 2π , where the raw series needed twenty rows for $h = 3$ and could not reach 4. The kernel carries eighteen, valid to $h = 4$ for $a \geq 20$ and to a reach that shortens linearly toward $a = 2\pi$, where the real and the periodic obstructions are equidistant. Below 2π the raw rows serve, seventeen of them for $\theta \leq 0.35$. The two regions meet at $a = 2\pi$, which is also the cell boundary of the table, so the whole book is one straight-line kernel with a single branch, three or four divisions and two square roots, and no two-dimensional table anywhere. On the tradeable 2024 S&P 500 feed it covers 29,997 of 30,000 quotes (91.1% through the raw rows, 8.9% through the conformal ones). The three it declines are extreme-wing quotes beyond that reach, and the region above $\theta = 0.35$ below 2π , which the feed never visits, stays with the large-volatility chart of Section 4.4. On a 1,200-point 40-digit truth set spanning both regions the worst error is 2.13 ULP in the raw region and 2.39 ULP in the conformal one, and the device transcription compiled for the host reproduces the CPU result bit for bit on the 832,287 quotes of the 849,818-point price box that the chart covers. Set against Table 6, where the wing at 1.17 ns on one quote in ninety is about a sixth of the routed book’s device cost and the tabulated chart carries a 200 KB table, a kernel that answers those quotes at the small-moneyness chart’s cost was the target, and it is met. On the same card the whole-book kernel inverts a book sorted once by a at 0.032 ns per quote, under the routed small-moneyness kernel alone. The entry point places this kernel in front of the routed charts. A quote outside its reach falls through to the routed inverter, so the total contract of Section 6 is unchanged and the kernel’s three declined quotes in thirty thousand cost the routed charts’ price. Its accuracy, its throughput against the routed charts and against the reference solver, and its bit-identity across builds, on the CPU and on the device, are reported in Section 7.

6 Routing and the bit-identical vectorized implementation

This section develops the runtime phase sketched in Algorithm 1, namely how each quote is routed into its regime and evaluated by kernels the scalar and SIMD paths share bit-for-bit. The offline generation is detailed in Appendix C and the per-chart kernels in Appendix B.

The design goal that shapes the implementation is that a batched evaluation must return,

for every quote, the same double-precision result as the scalar entry point, bit for bit. This is not a convenience. It means the vectorized surface and grid drivers can be validated by the single statement “batch equals scalar”, and it removes any possibility that the fast path and the reference path silently disagree. The invariant is achieved by writing every kernel as a branchless chain of *explicitly ordered* fused multiply-adds and sharing that exact code between the scalar and SIMD paths.

Three kernels are shared. A branchless logarithm proxy evaluates $\log_2 m$ for a mantissa $m \in [1, 2)$ through the reduction $t = (m - 1)/(m + 1)$ and a Chebyshev series in t^2 , with absolute error a few units in the last place, comparable to the library function it replaces. A full natural logarithm, needed by the universal inverse, the quantile seed and the wing, is then assembled by reuse as $\log x = \log 2 (e + \log_2 m)$ from the exponent e and mantissa of x , introducing no new approximation and agreeing with the library logarithm to three units in the last place over the domains that consume it. A subnormal-safe variant rescales arguments below the normal range by an exact power of two before the bit extraction, so the wing’s prices near 10^{-320} enter with full accuracy. A degree-seventeen branchless kernel evaluates the at-the-money base $v_0 = 2\sqrt{2} \operatorname{erf}^{-1}$. And a bivariate Chebyshev evaluator computes (7) by an inner accumulation in x_λ followed by an outer Clenshaw sweep in x_h . One design lesson from gating these kernels deserves record, because it is counterintuitive. For a kernel that feeds a downstream polynomial fit, the smoothness of its error function matters as much as its magnitude. An early candidate for the logarithm kernel, a plain minimax polynomial of the *same* worst-case accuracy as the reduction above, raised the Chebyshev degrees demanded by the table cells by eleven, because its equioscillating error injected a rough component into the quantity being fitted. The smooth-error reduction adds no degree at all. Kernel acceptance was therefore gated on the downstream fitted degrees, not only on the kernel’s own error.

Two more kernels serve the intrinsic chart of Section 5, and both were designed for the vector path. Its coordinate $a = \log(1 + q)$, $q = E/c$, needs $\log 1p$ to well under a unit in the last place because the chart’s sensitivity to a is 1.34. The classical route, a logarithm of the rounded $1 + q$ plus the Sterbenz correction $(q - d)/u$, costs a division and, with the library-grade $\log$ we first tried, 1.19 ULP against 80-bit truth. Reducing $1 + q$ into $[\frac{3}{4}2^k, \frac{3}{2}2^k)$ instead lets the reduced argument $f = (q - (2^k - 1))2^{-k}$ be formed from q itself, exactly by Sterbenz’s lemma for every $k \geq 1$ and trivially for $k = 0$, so the correction and its division disappear. The wider window needs an eight-term fit of the classical odd-argument polynomial, and the kernel measures 0.87 ULP worst against 80-bit truth over four million points, below the library’s own 0.81-ULP worst by a comparable margin. The whole-book kernel also needs $\operatorname{expm1}$ to a unit in the last place up to $h = 16.5$, which the small-moneyness Taylor kernel does not provide. Writing $h = k \log 2 + r$ with $k = \lfloor h / \log 2 \rfloor$ and $r \in [0, \log 2)$, the Cody–Waite split is exact, a twenty-term Taylor series of $\operatorname{expm1}(r)/r$ has only positive terms, and $E = 2^k \operatorname{expm1}(r) + (2^k - 1)$ adds two non-negative numbers, so the kernel is division-free and measures 1.00 ULP worst against the library over the whole range. Each kernel has an eight-wide AVX-512 twin and a four-wide AVX2 twin that reproduce the scalar fused-multiply order lane for lane. Masked selections use blends whose argument order is mirrored from the scalar ternaries, and per-lane routing decisions are the same integer tests as the scalar router. The only intrinsic without a direct AVX2 analogue is the exponent scaling **scalef** used by the shared exponential. We reproduce it, including its behaviour in the subnormal underflow range, by an exact two-factor power-of-two construction (splitting the integer exponent so that each factor remains a normal number), so the AVX2 path matches both the scalar **ldexp** and the AVX-512 **scalef** across the whole finite range. The equivalence was verified bit-for-bit over seven hundred thousand arguments down to the underflow threshold.

The batch driver is shaped by what a real workload contains. An end-of-day option book, re-expressed on the out-of-the-money branch (Section 7), is about 90% small-moneyness quotes. A general mixed-maturity batch is therefore dominated by one chart, and the driver bets on

that. It streams the (h, c) vector eight (or four) lanes at a time and, for each block, does three things at once. It classifies every lane by evaluating the three seam curves. It evaluates the small-moneyness kernel on *every* lane speculatively, and it stores, with a masked write, only the lanes that classified as small-moneyness. Those are the $\sim 90\%$, written in input order with no sort and no scatter. The minority that classified otherwise is compressed into a small deferred buffer and handed to a conventional counting-sort-by-cell pass over the remaining charts. Because the dominant chart never touches the sort, the mixed-maturity cost per quote falls close to the kernel’s own cost rather than to the cost of routing plus sorting plus gathering every quote individually.

Making the classification itself vectorizable required removing the last library calls from the router. The two seam prices consumed on the hot path, $C(h, 1.85)$ (tabulated versus large-volatility) and $\text{expm1}(h)$ (the small-moneyness input), are replaced by frozen low-degree Chebyshev fits, and the wing seam and the ceiling are moved onto the shared exp kernel. The scalar entry adopts the identical fits, so the classification is a handful of branchless polynomials that the scalar and SIMD routers evaluate to the same bits, and the route each quote takes is unchanged. Speculation costs the wasted small-moneyness evaluations on the deferred minority, far less than the sorting it removes on the majority, and it is gated. A strided sample of the batch’s moneyness picks the speculative driver only when the small-moneyness share is actually high, and otherwise falls back to the plain counting-sort-by-cell driver, which is faster on the wing- or table-dominated feeds a real book never produces. Both drivers return bit-identical results, so the dispatch changes only speed.

One further property of the speculative driver is worth recording, because it is invisible in an operation count and was measured only after the fact. Vectorizing eight quotes per register removes the routing branch but not the *serial* structure inside each chart. The small-moneyness path is a sequence of dependent Chebyshev recurrences (two seam polynomials, expm1 , the universal inverse’s logarithm and regime fit, the at-the-money kernel, V_2 , V_4), each a fused-multiply-add ladder whose latency one eight-lane group cannot hide. The machine is therefore idle at the arithmetic ports while the register file still has room, since the small-moneyness kernel occupies twenty of the thirty-two vector registers with no spill. Running several *independent* eight-lane groups through those prefix chains at once fills the gap. The subtlety is that this must be done to the prefix only. Interleaving the whole kernel, including the finisher whose bivariate Clenshaw holds a seventeen-term Chebyshev basis, exceeds the register file and buys nothing measurable (a percent, inside the noise). Interleaving the prefix while the finisher stays one group at a time is worth about twenty percent on the market-realistic feed. Nothing in the per-quote arithmetic changes—only mutually independent groups are co-scheduled—so the bit-identity invariant is preserved by construction and was verified unchanged. The four-wide instruction set is deliberately excluded. With sixteen registers it already spills at one group, so the same restructuring there would only add stack traffic.

Three further drivers complete the set. A fixed-moneyness batch driver for the case in which all quotes share one h (so the h -derived constants are hoisted out of the loop entirely) is the throughput ceiling. Note that this is a fixed-moneyness batch, not an expiry slice, since at one maturity h varies across strikes. A permuted-output variant returns results in bucket order with the permutation, saving the scattered store when the caller can consume sorted output. And a warm-start driver for streaming recalibration takes the previous variance of each (K, T) node as its seed, skips routing entirely, and applies three fixed Householder steps on the exact equation. Within its basin (previous $W \leq 40$, volatility moving up to a few percent) it returns the new root to within a few ULP rather than the last bit, because the half-ULP rounding of its working variables is amplified by the deep-wing cancellation ratio, so a caller needing full accuracy re-inverts cold—a documented API contract. For the live streaming regime, where snapshot-to-snapshot moves are sub-percent, a measured two-step variant reaches the same warm floor as three steps (verified for $|\Delta v/v| \leq 1\%$ at every $W \leq 40$. It degrades only from 2% moves

at deep W) and is guarded by the size of its own last correction, so a move outside the two-step basin flags itself and falls back to a cold inversion rather than returning a degraded result silently. A thin book-mode wrapper packages the pattern. Exchange strike grids are standardized, so a live book’s (K, T) node set is persistent, the caller carries only static log-strikes and previous variances, and re-deriving h under a new forward is one subtraction per node before the whole book warm-refines with no routing at all. In every driver the deep-wing bucket is split by the wing’s three evaluation regimes so each sub-bucket is lane-uniform, and the wing kernel has both an AVX-512 and an AVX2 twin. Two routes remain scalar by design. The exact at-the-money line $h = 0$ and the rare large- β rescue branch of Section 4.3. Both drop to the same non-inlined scalar entry the reference path uses, so the bit-identity statement covers them trivially.

The correctness of the vectorization is checked adversarially rather than assumed. Perturbing a single constant inside the vector kernels alone, by one part in 2^{40} , makes the batch result diverge from the scalar result on precisely the quotes routed through those kernels, while the unperturbed build shows zero divergences. This confirms the vector path genuinely executes rather than silently falling back to the scalar kernels and passing the identity check vacuously. The same check was run independently for the AVX-512 and AVX2 builds.

The bit-identity invariant turns out to constrain the *build*, not only the source, and the constraint is instructive enough to record as a finding. Under GCC’s default C++ contraction mode (`-ffp-contract=fast`) the compiler is free to fuse any multiply–add pair into a single fused operation with one rounding [10, 12, 24], and it exercises that freedom during middle-end optimization, *after* inlining, so which pairs fuse depends on the inlining context of the surrounding translation unit. We observed the same header, compiled at identical flags into two translation units of one program, produce two floating-point functions that disagree by one to five ULP on about seven percent of deep-wing quotes. A non-inlined entry point compiled to 4,427 instructions in the benchmark unit and 2,932 in a test unit, from identical source. Source-level care does not defend against this, because the divergence is the optimizer’s context-dependent choice. The robust discipline is the opposite default—contraction *off*, every intended fusion an explicit `fma`. Since the accuracy-critical chains here were written that way from the start, the flag costs nothing measurable, and it upgrades determinism from a per-build property to a portable one. Bitwise-reproducible floating point has mostly been pursued for reductions, where reassociation is the hazard [7]. The discipline here targets the complementary hazard, fusion, to the same end. On every validation feed of Section 7, the AVX-512, AVX2 and non-SIMD builds, compiled with either GCC 11 or Clang 14, return bitwise-identical doubles for every quote, scalar and batch paths alike, verified by byte comparison of the full output vectors, not by summary statistics. In production terms, a risk system replaying a batch or reconciling results across heterogeneous nodes and compilers obtains identical doubles in every case, removing a class of reconciliation noise that solvers with data-dependent iteration counts can exhibit.

The input contract. A reference implementation is defined as much by what it does with bad inputs as by its accuracy on good ones, because a live feed delivers crossed quotes, stale prices and convention violations as a matter of course, and the worst possible response is a plausible-looking number. The entry points therefore implement a *total* contract. Every representable pair (h, c) returns either a machine-precision result or a quiet NaN, and a checked variant reports a machine-readable status alongside, at or below intrinsic ($c \leq 0$), at or above the forward bound ($c \geq 1$), convention violation ($h < 0$ or NaN input), the out-of-domain deep corner of Section 4.3, and a near-saturation advisory for $v > 8$, where the $1 - c$ representability bound of Section 3 limits what any double-precision inverter can resolve. In that last case the best-effort result is still returned. A convenience wrapper accepts the industry-convention inputs (forward, strike, undiscounted price, expiry, call/put flag), performs the in-the-money to out-of-the-money parity flip internally, and returns the annualized volatility with the same status, which removes the most common integration error, a wrong normalization, from the caller’s responsibility.

7 Numerical results

Oracle and test sets. The reference is an independent `mpmath` inverter at 40 significant digits that inverts the given double-precision price c by a bounded log-space Newton iteration (not `findroot`, not a round trip). Accuracy is the relative error $|\hat{\sigma} - \sigma_{\text{oracle}}|/\sigma_{\text{oracle}}$, and we also report the worst-case error in units in the last place (ULP), defined as the number of representable doubles between the returned value and the oracle value rounded to double. The oracle carries 40 significant digits, so that rounding is correct unless the exact value lies within 10^{-40} relative of a rounding boundary. The worst points of every campaign were recomputed at 60 digits and returned the same double in every case. We score on a 150×110 grid over $h \in [0.02, 16]$, $\sigma\sqrt{T} \in [0.02, 8]$ (16,039 feasible points, the “regular” set), on a hand-built “stressed” set of deep-wing, near-intrinsic and small- h deep-OTM points, and on a 3,511-point chart-stress feed constructed to exercise every routing region and seam. Three inverters are scored. The first is the book kernel of Section 5 with the routed charts behind it, which is the entry point as shipped. The second is the routed charts alone. The third is the competitor, the latest public C++ implementation of Jäckel [14], downloaded unmodified from the author’s website (Appendix D), compiled with the flags of its own Makefile (`-O3 -DNDEBUG -ffp-contract=fast`) and called through its normalized entry point with $\beta = ce^{-h/2}$. We verified on the extreme cases that this normalized call agrees with the library’s direct price interface, so no artefact of the calling convention enters the comparison. Per the scoring rule of Section 3, no method is scored against the volatility that *generated* a test price. On our feeds that round-trip construction error reaches tens to hundreds of ULP in the wings, identically for all methods tested. All figures below are free of it.

Hardware, compilers and timing protocol. All measurements were taken on an Intel Core i5-1145G7 (Tiger Lake, AVX-512-capable, four cores) under Linux (Ubuntu 22.04, WSL2), with GCC 11.4 at `-O3 -ffp-contract=off -fno-fast-math -funroll-loops` and the instruction set selected per build (`-march=native` for AVX-512, `-mavx2 -mfma -mno-avx512f` for AVX2, both SIMD sets disabled for the scalar build). The Clang 14 cross-check uses the same flags. The timing protocol guards against dead-code elimination with a volatile accumulator. The definitive figures of Table 5 come from the same machine quiesced (mains power, no concurrent load, the process pinned to a fixed core), running each benchmark binary five times, each internal row itself a median over fifteen timing passes of eight repeats. The table reports medians across the five executions, whose spread is a few percent. Contraction is disabled for the reproducibility reasons of Section 6 (every intended fusion is an explicit `fma`), and `-ffast-math` is deliberately avoided because it would break the scalar/SIMD bit-identity. Loop unrolling, by contrast, is enabled. It duplicates loop bodies without altering any per-iteration floating-point operation, so it is bitwise-neutral (verified), and it recovers about ten percent that `-O3` alone leaves on the table by keeping the four highest-degree Chebyshev loops rolled. The floating-point environment is the IEEE-754 default: round-to-nearest-even, gradual underflow (no FTZ/DAZ), hardware `fma`. Bit-identity is claimed for the tested configurations (two compilers by three instruction-set builds on this host), not for arbitrary hardware. The development host is a virtualized laptop, so only the quiesced, pinned protocol above produces reported numbers. Comparisons against the reference are like for like in the following sense. Its four source files are compiled into the same binary as our headers, by the same compiler, at the flags of its own Makefile (`-O3 -DNDEBUG -ffp-contract=fast`, the release build its author ships) and the same instruction-set switch as the build under test, and every method is timed by the same loop over the same quotes in the same session, starting from the raw (h, c) pair. One asymmetry is inherent. Our kernels are header-only and inline into the timing loop, whereas the reference is called across a translation-unit boundary. A second build of every binary with link-time optimization on both sides, which lets the compiler inline the reference into the loop as well, moves the reference’s row

Region	Book kernel in front			Routed charts alone			Let's Be Rational		
	max rel	RMS rel	ULP	max rel	RMS rel	ULP	max rel	RMS rel	ULP
Near (small h)	4.0×10^{-16}	1.6×10^{-16}	3	6.7×10^{-16}	2.2×10^{-16}	4	5.5×10^{-16}	1.9×10^{-16}	3
Far	4.9×10^{-16}	1.3×10^{-16}	3	4.3×10^{-16}	1.2×10^{-16}	2	4.2×10^{-16}	1.2×10^{-16}	2
Wing (deep OTM)	5.3×10^{-16}	8.6×10^{-17}	3	5.3×10^{-16}	8.2×10^{-17}	3	degrades, see text		
Upper (large v)	4.3×10^{-16}	1.0×10^{-16}	3	4.3×10^{-16}	1.0×10^{-16}	3	5.1×10^{-14}	2.6×10^{-15}	426
All	5.3×10^{-16}	9.9×10^{-17}	3	6.7×10^{-16}	9.7×10^{-17}	4	—	—	—

Table 2: Relative error in σ against a 40-digit oracle on the regular set (16,039 points), worst, root-mean-square and worst in units in the last place. Neither of our columns has a point above 10^{-15} . The regular set is uniform over the whole box, so the book kernel answers only the 425 points inside its reach ($h \leq 0.35a$ below $a = 2\pi$, $h \leq 4$ above) and hands the rest to the routed charts. On those 425 its worst error is 4.9×10^{-16} , and on the market set of Table 4, which it answers in full, 4.9×10^{-16} against the routed charts' 8.5×10^{-16} and the reference's 5.2×10^{-16} . The reference, at its author's release flags, matches the routed charts on the tabulated and small- h regions but degrades in the near-intrinsic right endpoint and the deep wing (text).

by three percent and the ratios by less than it moves our own rows (Table 4), so the boundary is not what the tables measure. One difference between the two campaigns must be kept in view when reading ratios. The earlier campaign of Table 5 compiled the reference at the flags of this section, contraction off and assertions on, where it takes 227 ns per quote on the market feed. At its author's release flags, the build of Table 4, it takes 194, seventeen percent less. Every ratio against the reference quoted outside Table 5 and its discussion uses the release build.

Accuracy. Table 2 and Figure 4 give the result. Both of our columns hold below 10^{-15} at *every* point of the validation grid, the entry point as shipped with a worst relative error of 5.3×10^{-16} (3 ULP) and the routed charts alone with 6.7×10^{-16} (4 ULP), and a median at the level of a single rounding. Where the book kernel answers, it is more accurate than the chart it replaces, 4.0 against 6.7×10^{-16} on the small-moneyness region, because its rows are exact and its one table is a function of a single variable. On the interior and the small- h region the routed charts are indistinguishable from Let's Be Rational, confirming that the table construction reaches the same last-bit accuracy as a state-of-the-art iterative solver. The two methods separate only at the extremes (Figure 5). In the near-intrinsic right endpoint ($c \rightarrow 1$, $\sigma\sqrt{T} \rightarrow 8$) the reference solver loses up to 426 ULP (1,036 points above 10^{-15}) where the routed inverter stays at 3 ULP, because the transformed price $\tilde{c}$ of Section 4.4 keeps the objective well conditioned as $c \rightarrow 1$, where the reference's normalized objective is not. The subtraction $1 - c$ itself is exact by Sterbenz's lemma, so the loss is one of conditioning, not of representation. In the deep wing the two methods receive inputs of different quality. The reference's $\sqrt{FK}$ -normalized price $\beta = ce^{-h/2}$ is smaller than the forward-normalized c by the factor $e^{-h/2}$, so at large h it reaches the subnormal range, where doubles carry fewer significant bits, several hundred orders of magnitude before c does. Its accuracy there reflects that input conditioning rather than its iteration. The routed inverter, whose wing chart works on the logarithm of the intact price, inverts prices down to 10^{-320} at machine precision. The stressed set tells the same story, with 0 points above 10^{-15} for either of our columns (worst 3.7×10^{-16} with the book kernel in front, 6.9×10^{-16} for the routed charts) against a 3.4×10^{-14} right-endpoint tail for the reference. The book kernel's switches are boundaries where a polynomial approximation could peak between sampled points, so they were sampled on purpose. A campaign of 20,000 oracle points at 40 digits concentrates 2,500 each on both sides of $a = 2\pi$ at every scale of distance from 10^{-6} to 10^{-1} , on the $\theta = 0.35$ edge, within two percent of the conformal reach line, on the $h = 4$ cap, at $a \rightarrow 0$, at $a \rightarrow 700$ and at $h \rightarrow 10^{-6}$, with a uniform fill. The entry point's worst error over the twenty thousand is 5.7×10^{-16} (four ULP, root mean square 1.5×10^{-16}), no class exceeds 5.7×10^{-16} and no point exceeds 10^{-15} , and the forty worst points recomputed at

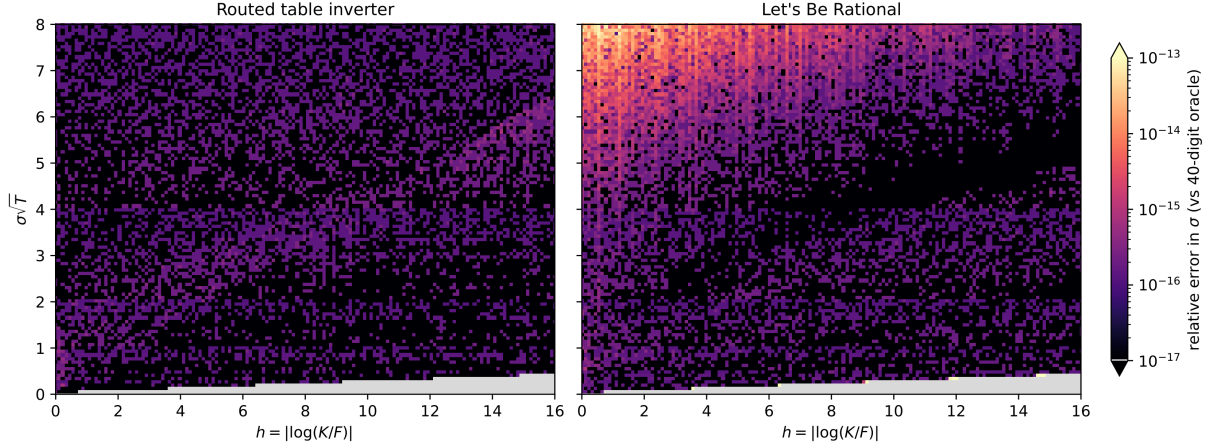


Figure 4: Relative error in σ against the 40-digit oracle over the $(h, \sigma\sqrt{T})$ domain. The routed table inverter (left) is uniformly at machine precision. Let’s Be Rational (right) degrades in the near-intrinsic upper-left corner and, more mildly, in the deep wing. Grey marks prices that round to 0 or 1 in double precision.

60 digits round to the same doubles. Across all validation sets combined the worst error the entry point has exhibited anywhere is that 5.7×10^{-16} , and the worst of the routed charts alone 9.2×10^{-16} , on the same campaign. No set has produced a point above 10^{-15} for either. Set against the propagated input rounding of Section 3, at most 1.3×10^{-16} on the box a traded book occupies, these are the whole error budget of a market inversion. At the $v = 8$ edge of the grid the propagated floor of 10^{-13} dwarfs both. The seam crossings between charts were additionally sampled densely on both sides. No error excursion occurs at any of them. This was not automatic. Dense seam verification exposed two seam placements whose corners violated the gate, and both were corrected (one by lowering the ceiling shared by the two tabulated charts, one by extending the wing crossover), the intended operation of an oracle-gated design.

Honesty about where the accuracy advantage does and does not matter is essential here. The two regimes in which the reference degrades—the near-intrinsic upper endpoint and the subnormal-price wing—are extreme. They require normalized prices within a rounding of one, or below 10^{-40} . A tradeable option book reaches neither. Inverting an independent 30-digit oracle on 1,150 *actual* projected prices sampled from the 2024 S&P 500 book (described below), stratified to over-weight the deep-cheap and seam-band quotes, the book kernel answers every one of them and has no point above 10^{-15} (worst 4.9×10^{-16} , RMS 1.6×10^{-16}), the routed charts alone reach 8.5×10^{-16} , and the reference 5.2×10^{-16} . On real market data, in other words, all three solvers are at machine precision, and the case for the present method there rests on throughput and determinism, not on accuracy. The accuracy advantage is real but it is insurance. It guarantees the same last-bit behaviour on the theoretical, stressed, and finer-tick inputs that a robust production system must also survive, without the reference’s two localized failure modes.

Edge cases. Table 3 isolates individual extreme inputs, with the exact double-precision price each method was given and the relative σ error each returned. The routed inverter is exact or within four ULP on every row, including prices in the subnormal range near 10^{-321} , the formerly excluded small- h deep-OTM corner, the near-intrinsic upper endpoint, and the feasibility pinch at $h = 16$. The reference solver’s two visible losses have different characters and should be read differently. Near the intrinsic value its error (10^{-14} scale) is algorithmic, the conditioning of its normalized objective discussed in Section 3. In the subnormal wing its input $\beta = ce^{-h/2}$ has already lost significance before the algorithm starts, so the error there (10^{-6} to 10^{-4}) reflects the

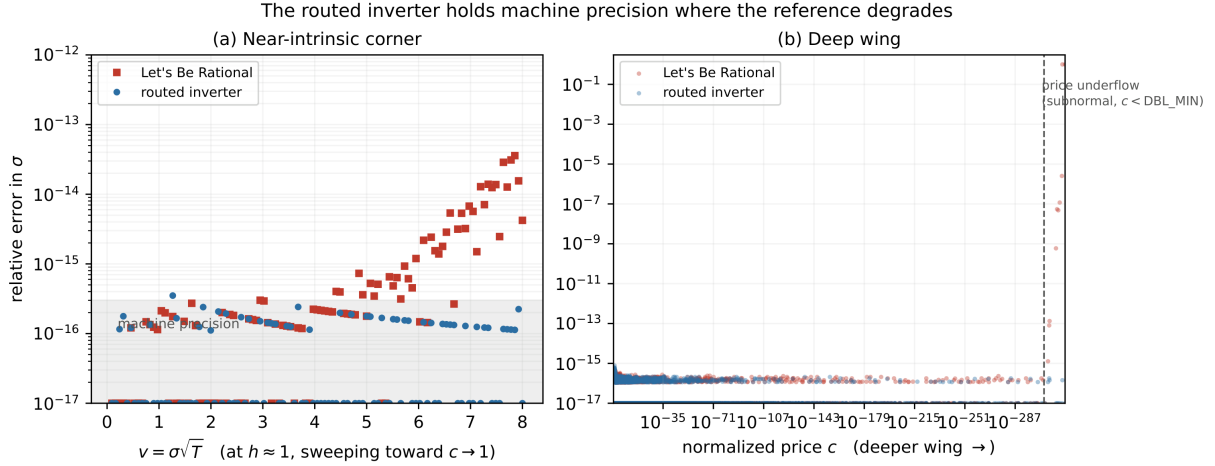


Figure 5: Relative error in σ against the 40-digit oracle in the two regimes where the reference degrades. **(a)** Near the intrinsic value: at fixed $h \approx 1$, as v grows and the normalized price $c \rightarrow 1$, Let’s Be Rational climbs toward 10^{-13} from the conditioning of its normalized objective near the upper bound while the routed inverter stays at machine precision. **(b)** In the deep wing, error against the price magnitude c over the whole wing chart. Both solvers hold machine precision across three hundred orders of magnitude, but once the price is subnormal ($c < \text{DBL_MIN}$) the reference’s reduced variable $\beta = c e^{-h/2}$ underflows and it loses precision, whereas the routed inverter, working in $c = \text{price}/F$, does not.

conditioning of its chosen normalization rather than its iteration. The routed inverter is immune because its normalization keeps the small price intact and its wing chart works on the logarithm of the price. At the two most extreme prices of the regular grid, below 10^{-320} , the β input underflows to exactly zero and the reference necessarily returns zero volatility, while the routed inverter still recovers σ to the correctly rounded last bit. Prices this small are admittedly of more diagnostic than trading interest, but they mark where each formulation’s validity actually ends.

Coverage and bit-identity. On a broad random feed of 120,000 quotes ($\sigma\sqrt{T}$ up to 8, h up to 16.5) every quote routes to a chart and returns a gated result, with no routing failures and no silent fallbacks. The continuum contract remains the graded one of Section 3. Across the scalar, AVX2 and AVX-512 builds and all batch drivers (fixed- h batch, mixed- h grid, permuted-output, warm-start), the batch result equals the scalar result on every one of the 120,000 quotes, with zero mismatches. The same holds on the 16,039-point accuracy grid. Beyond within-build identity, the full output vectors of the three instruction-set builds under GCC and of the AVX-512 and AVX2 builds under Clang were compared byte for byte on the chart-stress feed. All five binaries return identical doubles for every quote, in both the scalar and the batched path. The accuracy of Table 2 is therefore simultaneously the accuracy of every batch path, every instruction set, and both compilers.

What a real book exercises. Which charts does a realistic workload actually hit? We took one full year (2024) of end-of-day S&P 500 index option quotes from a standard vendor file (1.85 million with positive volume and open interest), and applied a tradeability screen: a live, uncrossed two-sided market, mid price at least five cents, relative bid-ask spread at most 25%, open interest at least 100 contracts and volume at least 10, and at least a week to expiry. This leaves 641,072 tradeable quotes. Forwards are the vendor’s own, matched to each quote by expiry *and* settlement convention (the file carries distinct AM- and PM-settlement forwards, differing by about half an index point), and discount factors come from the vendor zero curve.

h	$\sigma\sqrt{T}$	price	regime	this method	Let's Be Rational
14.8	0.386	$c = 9.9 \times 10^{-321}$	wing	0	1.4×10^{-4}
3.56	0.0932	$c = 2.2 \times 10^{-321}$	wing	0	3.9×10^{-6}
8	0.90	$c = 1.5 \times 10^{-18}$	wing	1.2×10^{-16}	0
0.29	0.02	$c = 9.6 \times 10^{-51}$	corner	0	0
10^{-4}	1.0	$c = 0.38$	left	0	0
0	6.4	$c = 0.9986$	ATM	5.6×10^{-16}	1.4×10^{-16}
0.5	7.9	$1 - c = 1.0 \times 10^{-4}$	right	1.1×10^{-16}	5.6×10^{-16}
1.2	7.93	$1 - c = 1.3 \times 10^{-4}$	right	1.1×10^{-16}	1.3×10^{-14}
16	7.5	$c = 0.93$	right	1.2×10^{-16}	1.2×10^{-16}

Table 3: Relative error in σ against the 40-digit oracle on hand-picked extreme inputs. An entry of 0 means the returned double equals the correctly rounded oracle value. The reference solver’s subnormal-wing losses reflect the conditioning of its $\sqrt{FK}$ normalization (text). Its near-intrinsic losses arise in its own normalized variable near the upper price bound, where its objective conditions poorly—an observed property of that implementation, distinct from the representability bound of Section 3, which binds both methods equally.

We do not rely on a parity regression, which we found can bias the implied forward by up to several tenths of a percent at individual expiries where deep in-the-money premiums dominate the fit. Every surviving quote (call or put, in- or out-of-the-money) is then folded onto the single out-of-the-money-call branch by put–call parity, exactly the reduction the inverter itself uses. 15.8% of quotes are in-the-money and reach the branch through the parity projection. Routing the resulting (h, c) pairs gives: small-moneyness chart 92.0%, tabulated chart 6.8%, deep wing 1.2%, large-volatility chart 0.0% (Figure 2). Three facts deserve emphasis. The wing, though barely one tradeable quote in eighty, is a permanent feature of this book—97% of those quotes are downside puts, the crash-skew tail [11] firing the wing route on absolute cheapness—which is what motivated vectorizing it rather than leaving it scalar. The large-volatility chart never fires on end-of-day equity-index data. It earns its place as coverage, not throughput. And the maximum log-moneyness observed all year is $h = 3.4$, far inside the design domain. The uniform broad box used for coverage stress, by contrast, is 70% wing—a distribution no traded book resembles—so throughput is reported on a market-realistic feed, built by resampling 30,000 quotes from this tradeable population, which reproduces the live route mix.

Throughput. Table 5 and Figure 6 report median nanoseconds per quote for the reference solver, the routed inverter’s scalar entry, and its batch drivers on both instruction sets. One qualification governs every row. The reference is timed as its author’s official public *scalar* implementation, a reading the limitations paragraph makes precise. The gain is nevertheless genuinely width-driven. The AVX2 build recovers roughly half of the AVX-512 advantage on every workload. So that no method is flattered by the harness, the market-feed row is timed *all-inside*. Every method starts from the raw (h, c) pair within the timed loop, so the reference pays its $\beta = ce^{-h/2}$ input transform and our scalar entry builds its per-quote context. The fixed-moneyness rows, by contrast, isolate the kernels by hoisting the shared per- h constants, so they are chart-pure kernel benchmarks rather than expiry slices, since at one expiry h varies across strikes. Three patterns stand out. On the chart-pure fixed-moneyness batches the batch driver is several times faster than the reference, $5.4\times$ on the small-moneyness chart, $9.9\times$ on the tabulated chart, and $2.8\times$ on the compute-heavy large-volatility endpoint (on AVX-512, with AVX2 recovering 3.3, 9.1, and $1.4\times$). This is the direct consequence of a fixed, branch-light operation count—eight or four quotes advancing through the identical polynomial sweep in lockstep, where a branch-rich iteration cannot. On the *market-realistic feed*, the workload that matters, the speculative driver of Section 6 inverts a mixed S&P 500 book at 48 ns per quote

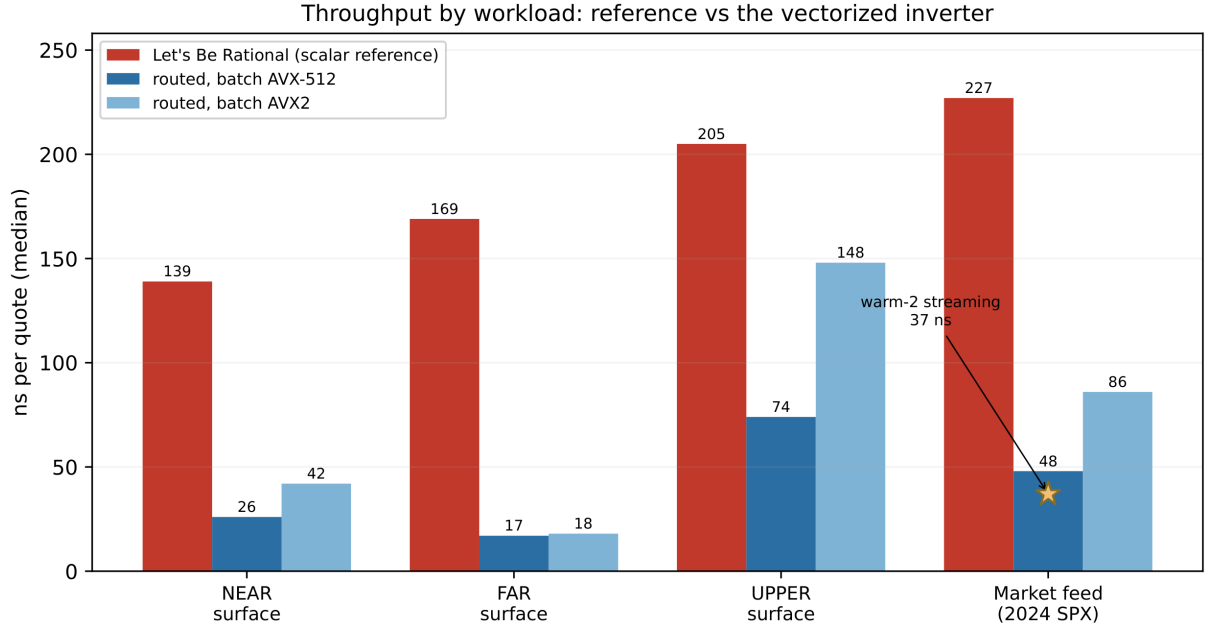


Figure 6: Median nanoseconds per quote by workload (the data of Table 5), for the scalar reference solver and the routed inverter’s two vectorized batch paths (AVX-512 and AVX2). The batch paths are several times faster than the reference on the chart-pure fixed-moneyness batches and on the market-realistic feed. The star marks the warm streaming steady state on the market feed (37 ns, AVX-512. Few-ULP warm accuracy contract).

against the reference’s 227, a factor of 4.7. On AVX2 it is 86 against 225, a factor of 2.6. Against the reference’s release build in the later one-binary campaign (Table 4) the same driver’s factors are 4.3 and 2.2. The small-moneyness chart carrying 92% of the book at close to its kernel cost is what carries the result. The deep wing deserves separate mention, because it is where the comparison has moved. Its evaluator carries more arithmetic than a Householder iteration, and on a wing-weighted grid the reference used to lead the batch driver by half again. Seeding the wing’s Newton solve from the tabulated surface of Section 4.3 closed that gap. On the wing subset of the broad grid the vectorized driver now inverts at 250 ns per quote against the reference’s 307, a clear lead rather than a deficit. The scalar entry remains far behind there (1,225 ns), which is the cost of a fixed-count kernel shared with its SIMD twin, so a caller inverting deep-wing quotes one at a time should still prefer the reference. Since only about one quote in eighty of the book studied is a wing quote, neither figure moves the aggregate much. What changed is that no chart is now a batch-mode liability.

One throughput property is invisible in medians but operationally significant. The cost per quote is *deterministic*. Every chart executes a fixed operation count (a fixed-degree polynomial sweep, or a fixed number of refinement steps, three seeded Newton in the wing, three Householder at the large-volatility endpoint), so the routed inverter has no iteration-count distribution and no data-dependent tail latency. The slowest quote of a batch costs the same as the median quote of its chart. The reference solver is also fixed-count (a seed and two Householder corrections), so its cost distribution is likewise narrow. The distinction here is not the absence of a tolerance loop but that the fixed count is executed by *branchless* kernels, which is what lets eight quotes advance in lockstep. Solvers that do iterate to a tolerance carry a tail, however short, and hedging pipelines that wait on the last quote of a batch inherit it.

The book kernel. Table 4 times the book kernel of Section 5 on the same market feed, in one binary with the routed charts and the reference. Two campaigns feed it. The first, on the

quiet host under the protocol above (five pinned repetitions, spread under one percent on every row), times the book kernel against the routed charts. Over the full feed with every route the kernel’s batch path takes 28.6 ns per quote on AVX-512 and 40.7 on AVX2 against 43.3 and 83.0 for the routed speculative batch, 1.5 and 2.0 times. At batches of 64 quotes, where the routed driver’s sort and drain do not amortize, the ratios widen to the 2.5 and 3.5 of Table 4, and the kernel’s scalar entry takes 118 ns against the routed scalar entry’s 263. The second campaign, under the same quiet protocol, adds the reference and the PDE table method of Matić et al. [22] to the binary, each built from its authors’ sources at its own release flags. The reference’s scalar rate, 194 ns per quote, is 6.7 times slower than the kernel’s AVX-512 batch and 3.8 times slower than its AVX2 batch, and 1.6 times slower than the kernel’s scalar entry on the same quotes. The last of these changes the single-quote picture. The routed charts’ scalar entry, whose routing and per-quote context were designed to amortize across a batch, is slower than the reference on isolated cold quotes, as recorded below. The book kernel’s scalar entry, which routes nothing and iterates nowhere, is faster than the reference on the book on both vector-capable builds, so a caller inverting one quote at a time no longer gives anything up. On the build without SIMD and without hardware fused multiply-add, the last row of the upper block of Table 4, every explicit fma of our kernels becomes a library call and the reference, which has none, is the faster scalar there. That row is the cost of the kernels on hardware the design does not target. The whole gain is arithmetic. The kernel executes about three hundred fused multiply-adds, three or four divisions and no transcendental library call, in one straight line, whereas the reference’s two Householder corrections each evaluate a carefully written Black function with its error-function calls, and the routed charts pay two seam polynomials and a sort on top of a kernel of comparable size. On the small-moneyness quotes alone, where the routed chart is at its best, the eleven-row recurrence form of the kernel takes 16.6 ns on AVX-512 and 23.9 on AVX2 against the routed chart’s 23.4 and 62.4, and the whole-book kernel’s seventeen rows and second table cell cost 25 on the same tile in the one-binary campaign (Table 7), about eight nanoseconds more, the price of covering the interior and the wing in the same straight line. For completeness, the vectorized port of the reference by Saqr [26], run on the same feed and host from Python with its Numba backend at one thread, takes 411 ns per quote (whole batch of 30,000, median of seven passes, 350 with all eight threads), with a worst relative error of 3.3×10^{-14} . Its default Halley path takes 2,160 ns at a worst error of 6.8×10^{-5} . Those figures were taken on a loaded session rather than under the quiet protocol and should be read to one significant figure. The library’s author reports 116 ns per quote for the same path on a 10^5 -point grid of his own. Either figure places a whole-array Householder iteration, masked across four branches, a factor of two behind the scalar reference in compiled C++ rather than ahead of it.

Streaming re-inversion. A live system rarely inverts cold twice. The persistent node set of Section 6 means every snapshot after the first starts from the previous one. On the market feed, seeding each node with its previous variance after a half-percent move, the guarded two-step warm batch completes in 0.76 times the cold speculative driver’s time on the same feed and host—37 ns per quote against the cold driver’s 48 and the reference’s 227. That $6.2\times$ margin (5.2 against the release build of Table 4) is a deployment-mode number, not a method-to-method one—the warm driver is handed each node’s previous root, which the cold reference is not, and delivers the warm few-ULP contract—but it is the number a persistent-book system experiences. Its advantage over a cold re-inversion has narrowed as the cold driver improved. Warm was worth about twice the cold path before the driver was interleaved, and is worth roughly 1.3 times now. The streaming case for it rests less on raw margin than on the shape of the cost. Having no routing and no deferred drain, its cost is flat in batch size from one full register upward (Figure 1), whereas the cold driver’s sort-and-drain overhead only amortizes over batches of a few thousand. Below one register width the warm driver, like every batch driver here, falls to its scalar tail, so the figure’s step at eight quotes is the register filling and not a measurement artefact: at one, two

Build	Let's Be	PDE table	Routed charts		Book kernel	
	Rational	method	scalar	batch	scalar	batch
AVX-512, full feed	194	76	289	45	118	29
AVX-512, batches of 64				75		30
AVX2, full feed	193	86	298	89	142	51
AVX2, batches of 64				151		43
no SIMD, full feed	220	89	557	578	342	352
AVX-512, full feed, LTO on every side	189	82	267	43	118	29
AVX2, full feed, LTO on every side	188	85	269	83	119	42

Table 4: Nanoseconds per quote on the market feed (30,000 actual projected prices, every route), all methods in one binary, timed by the same loop in the same session on the quiet host of Table 5 (median of seven timing passes of four sweeps, with run-to-run spread of the whole campaign under one percent), every method starting from the raw (h, c) pair. The reference is compiled from its author’s sources at the flags of its own Makefile. The PDE table method of Matić et al. [22] is compiled from its authors’ sources at the flags of its compile script, and timed as its scalar evaluation of the 46 MB table, single-threaded, whereas its own CPU convention uses every core under OpenMP. The routed charts and the book kernel are at the flags of Section 7. The batch columns are the speculative driver of Section 6 and the kernel’s lane-for-lane SIMD twin. On the no-SIMD build both degenerate to their scalar entries, which is the honest cost of the kernels without a vector unit. That build also lacks hardware fused multiply-add, so every explicit fma in our kernels is a library call there, which the reference does not pay. The lower block rebuilds every binary with link-time optimization on every side, so that the reference and the PDE method are inlined into the loop as our headers are. It moves their rows by three percent and eight percent, less than it moves ours on AVX2, so the translation-unit boundary is not what the table measures. Bold marks the book kernel’s batch path. Its output is bit-identical to its scalar entry on every quote, verified inside every timed binary.

and four quotes the warm path costs 288 ns per quote against the reference’s 264, and at eight it costs 37. The counterpart concerns the single isolated quote. Inverted scalar and cold, one quote at a time, the reference solver remains faster than the routed charts’ scalar entry, whose per-quote context and routing were designed to amortize across a batch. The book kernel’s scalar entry, which has neither, is 1.6 times as fast as the reference on the same quotes (Table 4), so with the kernel in front the isolated cold inversion is no longer a workload the reference wins. The warm contract can now be stated more sharply than a sweep allows. The update actually run is Householder’s method of order three applied to $F(v) = C(h, v) - c$. Written in $x = v/2$ on $g(x) = \frac{1}{2}e^{-h/2}(c - C(h, 2x))$, the constant prefactor and the affine change of variable both cancel out of the Newton quotient and of the two curvature ratios, so no translation is lost. Feeding the exact derivatives $F' = \phi(d_1)$, $F'' = -d_1\phi(d_1)(k/v^2 + \frac{1}{2})$ and their two further v -derivatives into a Newton–Kantorovich gate on a disk about the previous root, and replacing the Newton majorant by the degree-three one appropriate to an order-four iteration, gives a per-quote certificate that is elementary to evaluate and was validated against the exact iteration at ninety digits on 829 gated cases with no violation. Three consequences matter here. The two-step fast path is a theorem for $W \leq 4.33$ and remains sweep-measured beyond it, Kantorovich being sufficient rather than necessary. The certified move over the whole code box is a (h, W) -indexed surface rather than a single percentage, and reading a single number off it understates the shallow end by a factor of six and overstates the deep end by a factor of two and a half. And, most usefully, the bound at $W = 8$ under a half-percent move is 1.5×10^{-17} , some 160 times below the floor actually observed, which turns this section’s attribution of that floor to the double-precision price representation, rather than to the iteration, from an inference into a consequence.

Workload	Let's Be	Routed inverter, ns per quote		
	Rational	scalar	batch AVX-512	batch AVX2
NEAR batch, $h=0.2$	139	141	26	42
FAR batch, $h=1$	169	69	17	18
WING subset [†]	307	1225	250	347
UPPER batch, $h=1$	205	557	74	148
Market feed (2024 SPX, actual prices)	227	321	48	86
Market feed, warm 2-step (streaming)	227	—	37	79

Table 5: Median nanoseconds per quote (five repetitions, quiet Intel Core i5-1145G7, Linux, GCC 11.4, `-O3 -ffp-contract=off -fno-fast-math -funroll-loops`), on chart-pure fixed-moneyness batches of 4096 quotes (fixed h , varying price, so kernel benchmarks rather than expiry slices), and on the market-realistic feed of 30,000 actual projected prices resampled from the tradeable 2024 S&P 500 population (route mix 92/1/7/0 percent across near/wing/far/upper. Mixed h , the realistic book workload). The reference column is the author’s official public scalar implementation compiled at the flags above, contraction off and assertions on. At its author’s release flags it is seventeen percent faster on this host (Table 4). The market row is timed all-inside (each method pays its own per-quote input transform inside the loop). The fixed-moneyness rows hoist the shared per- h constants. The batch output is bit-identical to the scalar entry on every driver and instruction set (zero mismatches, verified on both the 120,000-point coverage grid and the market feed). [†]The wing row is measured differently from the three above it and the two below, and the difference is a property of the drivers rather than of the chart. The fixed-moneyness driver hoists per- h constants but evaluates wing quotes through the scalar entry, so it cannot report a vectorized wing. The mixed-grid driver does vectorize the chart, in regime-uniform buckets. The wing row therefore reports the wing subset of the 120,000-point coverage grid under the mixed-grid driver, which is also why its reference column differs from the fixed-moneyness rows. It is a different, wing-only population. The final row is the streaming steady state, the guarded two-step warm driver on the market feed, seeded from each node’s previous variance, under the few-ULP warm contract rather than the last bit (Section 6). It is listed against the reference for scale, not as a like-for-like comparison. Bold marks where the batch beats the reference. Inter-repetition spread was a few percent on every cell.

The same algorithm on a GPU. The bit-identity discipline was motivated by CPU SIMD, but its payoff is larger. Kernels whose entire rounding behaviour is a frozen, branchless fused-multiply-add sequence port to a GPU with their semantics intact. We mirrored all four charts as CUDA device kernels, operation for operation (compiled with `-fmad=false`, the device analogue of `-ffp-contract=off`), and scheduled them the way a persistent book invites. The book is stored *bucket-ordered*, partitioned by chart, each chart runs as one kernel over a contiguous slice, and no sort, gather, or scatter touches the device in the timed passes. The figures below are therefore resident, pre-partitioned kernel throughput. How far a partition persists across snapshots is measured separately at the end of this section. The host performs only the per- h hoists the CPU index pass already performs, and the rare scalar-only branches (the rescue wedge, the analytic edges. Zero quotes on this feed) stay host-side, counted and excluded rather than silently mishandled. On the market feed tiled to 5.01 million quotes, every device result equals the CPU scalar entry’s double *exactly*, with zero mismatches at zero ULP, on every chart, in every run. The same holds on four chart-pure fixed-moneyness surfaces of 5.24 million quotes each, which is also how the large-volatility chart is exercised at all, since the tradeable feed routes nothing to it. The determinism claim of Section 6, five CPU builds returning identical bits, extends verbatim to an architecturally alien execution engine. Table 6 reports the throughput, 0.078 ns per quote for the full market mix on one NVIDIA H100 with the routed charts, or one

million quotes in under eighty microseconds, and 0.032 with the whole-book kernel of Section 5 on a book sorted once by its intrinsic coordinate, one million in thirty-two. Two qualifications govern the reading. The figure is kernel-resident (the book lives on the device, as in a persistent deployment, and host transfers are excluded), and it is fp64 throughput on datacenter-class hardware. Consumer GPUs execute double precision at 1/32–1/64 rate and will not reproduce it.

A number of that size invites disbelief—a tenth of a nanosecond is roughly the time light needs to cross the die—so it is worth showing that it is unremarkable. It is a throughput, not a latency. No individual quote is inverted in that time, and a single isolated query costs microseconds dominated by launch overhead. But of order 10^5 threads are resident at once, so completions emerge at that rate. The card carries 114 multiprocessors of 64 double-precision lanes each, 7,296 in all, at about 1.7 GHz, against the development laptop’s single 512-bit fused-multiply-add port, eight lanes, at about 3.2 GHz. The ratio of double-precision execution resources is therefore of order 400 to 500, the range reflecting how far the laptop’s clock throttles under sustained wide-vector load. The measured ratio between the two market-feed figures is about 620, modestly above that range, and the excess is informative rather than anomalous. The CPU figure is a *cold* driver that routes, sorts and drains on every snapshot, whereas the device schedule stores the book bucket-ordered once and never re-partitions it, so the comparison charges the host for work the device has been designed out of. Set against the CPU’s warm streaming driver, which likewise does no routing, the ratio is about 470 and falls inside the range. Both implementations thus attain broadly the same efficiency per execution unit, which is the substantive point. The device figure is the hardware’s arithmetic rather than an artifact of the harness, and the routed method carries to a machine three orders of magnitude wider without losing efficiency on the way. The whole-book kernel tells the same story with one addition. Its CPU batch at 28.6 ns against its device figure in file order, 0.072, is a ratio of about 400, at the bottom of the range, as it should be, because the AVX-512 twin evaluates both regions on every lane and blends, so it pays on the CPU what a mixed warp pays on the device. Sorting the book by *a* removes that cost on the device, where lanes are grouped by warp, and not on the CPU, where a vector of eight always mixes. The sorted 0.032 is a ratio of about 900, and the excess over the resource ratio is exactly the divergence a sorted book no longer pays.

One device-side lesson generalizes beyond this solver, and it is the mirror image of the register argument that shaped the CPU driver. The wing initially ran twenty-four times slower per quote than the small-moneyness chart, which no operation count explains. The cause was storage class, not arithmetic. Its $\log H$ fit accumulated a row of Chebyshev coefficients and a row of derivative coefficients into two arrays indexed by a *runtime* loop bound, and an array a compiler cannot resolve to registers is placed in device *local* memory—off-chip. The compiler’s own report made this visible without running anything, showing a 432-byte stack frame, exactly the two arrays, and 128 registers per thread, which caps occupancy near a quarter and removes precisely the warp-level parallelism that hides latency. The small-moneyness chart, whose polynomial degrees are compile-time constants and therefore fully unrolled into registers, reported a zero-byte frame and thirty-two registers. The fix is a restructuring, not new mathematics. Because both outer Clenshaw recurrences and the derivative recurrence run in the same direction, each row can be computed and consumed in a single backward sweep, holding a constant number of values instead of two arrays. The frame falls to zero bytes, and in the campaign where the change was made the wing evaluated in 0.83 instead of 1.28 nanoseconds on the feed’s wing slice, taking the full book from 0.12 to 0.10. Every lane still executes the identical sequence of fused multiply-adds, so the transformation is bitwise neutral, and it was verified as such against the array form over the full feed and a dense wing sweep. The wing is no longer the device’s cost centre. Its per-quote cost in situ has since risen to 1.17 ns, because the seam of Section 4 now hands all but the genuinely extreme quotes to the tabulated charts and what remains sits deeper, but it carries only 1.1% of the book and so accounts for about a sixth of the total. Two thirds is now the small-moneyness

chart, 92% of the book at 0.057 ns, and that is where the next lever had to act. The kernels of Section 5, below, are that lever.

A second layout lesson, opposite in sign to the first, shaped Section 5. A first device port of an oracle-gated two-dimensional table in the intrinsic coordinates, twelve cells with a graded row profile stored as ragged rows behind three offset arrays, measured 0.062 ns per quote on the same H100 against the small-moneyness kernel’s 0.037 while executing *less* arithmetic, about sixty-four fused multiply-adds against two hundred and fifty. Launching one kernel per cell over bucket-ordered slices, the schedule this paper recommends for the shipped charts, made it slower still, 0.077, because twelve launches cost more than the divergence they removed. The loss lay in the table’s layout. Its loop bounds are data-dependent, so the compiler cannot unroll them, and its coefficients are reached through an index rather than present as constant-bank operands, whereas the shipped kernel is straight-line code whose every coefficient is a compile-time array index. Emitting the row profile as template constants restored that shape and, with it, register parity with the shipped kernel (36 registers, no stack frame). The exact rows of Section 5 then removed most of the table altogether. The compiled recurrence kernel is 512 machine instructions with 134 double-precision fused multiply-adds, against 696 and 190 for the template-emitted table it replaced, in straight-line code. The whole-book kernel compiles to 48 registers with no stack frame. Both reproduce the CPU reference bit for bit when their device functions are compiled for the host over the full price box, the port check the shipped kernels passed before their device runs, and on the device itself (Table 6, lower block). Their timings confirm the layout lesson and add one of their own. The recurrence kernel inverts the small-moneyness feed tile at 0.021 ns per quote against the shipped small-moneyness kernel’s 0.037, the same in file order and sorted, since it has no branch to diverge on. The template-emitted table kernel takes 0.036, and the ragged twelve-cell table, on the same tile in file order, 1.50, twenty-four times slower than its bucketed 0.062, which is the divergence a data-dependent layout costs when its cells are not warp-uniform. The whole-book kernel carries one branch, at $a = 2\pi$, and region B’s two square roots. In file order, where nine percent of the feed is region B and every warp mixes the two, it inverts the full feed at 0.072 ns per quote, seven percent under the shipped bucket-ordered book. On the same full feed sorted once by a it takes 0.032, well under half, and below the shipped small-moneyness kernel alone, with both device runs agreeing to three digits. The shipped 0.078 is itself a sorted schedule, the book partitioned once by chart and kept so, and a book kept partitioned by the side of $a = 2\pi$ is the corresponding deployment of the whole-book kernel. Whether such a partition persists is an empirical question, and the vendor file answers it. Over the 43,485 nodes of the year’s tradeable quotes, observed on 596,566 pairs of consecutive quoting days, the intrinsic coordinate of a node moves by 0.20 per day at the median, so a full sort by a is not persistent. But only the side of $a = 2\pi$ matters for warp uniformity, and that side changes on 2.2% of the node-days, the routed charts’ membership on 0.9%. A partition kept from the previous snapshot therefore leaves about two percent of quotes in warps of the other region, which pay the file-order penalty. The size of that effect, and the cost of the alternative, a per-snapshot one-bit partition that is a linear pass over the book, have not been measured. The file-order figure is the unconditional one, and the partitioned figure is stated for that regime. Branch by branch on region-filtered tiles (Table 7) the kernel costs 0.070 on the small-moneyness tile, 0.087 on the interior tile, where region A and B quotes share warps, and 0.045 in the wing, where the conformal rows run without a branch. With the inputs uploaded from pageable host memory and the results read back on every pass, the same kernel takes 1.12 ns per quote. The difference is the 120 MB of host traffic per pass, and it is the convention under which the table method of Matić et al. [22] reports and under which Section 7 compares the two.

Relation to other methods. The natural axes of comparison are accuracy class, domain of validity, and unit of work. The explicit formulas of Li [19], Stefanica and Radoičić [29] and

Xia and Cui [31] are the fastest possible single evaluations and vectorize trivially, but they are approximations valid on bounded regions with accuracy far from the last bit. In the present architecture their natural role is as seeds, and the rational quantile seed of Section 4.4 plays exactly that part. Matić et al. [22] precompute a polynomial table over moneyness and price, as we do, but target an accuracy of 10^{-7} . Their open-source C++/OpenCL implementation ships a roughly 46 MB coefficient file against the present table’s 200 KB. The gap in accuracy and size has one source. Removing the singular endpoint regimes from the two-dimensional fit lets the tabulated chart follow the branch-point principle of Section 4.2 instead of refining toward the boundaries. They also report timings, 9.25 ms per million prices on a Tesla V100 at their 10^{-7} tolerance, and their code is public, so we ran it on our feed and our card. Built from their sources at their flags, with their 46 MB table, and driven by the same 30,000 tradeable quotes tiled to five million, their OpenCL path (double precision) takes 7.68 ns per quote on the H100 in their own convention, which uploads the inputs and reads the results back on every call, and 4.93 with the uploads removed. The whole-book kernel under that same transfer-inclusive convention takes 1.12 ns per quote, and kernel-resident 0.032 on the book sorted by a (Table 6). Their 7.68 on the H100 against 9.25 on the V100 shows their figure is bound by host transfers rather than arithmetic, as ours is under the same convention. On the CPU their single-threaded scalar evaluation takes 76 ns per quote against the book kernel’s 118 scalar and 29 batch on the quiet host of Table 4. Their published CPU convention uses every core under OpenMP (11 ns per quote on a twenty-eight-core server), where every figure of Section 7 is one thread. On accuracy the two are not in the same class. On the traded feed their worst relative error is 1.2×10^{-5} (near the money at $h = 1.7 \times 10^{-4}$, mean 2.8×10^{-9}) against 4.9×10^{-16} for the book kernel. Table 7 gives both methods branch by branch on identical region-filtered tiles of the feed, so that neither the route mix nor the tile favours either side. Their cost is nearly flat across branches, as a single table’s must be, and the book kernel’s batch path is 1.7 to 3.0 times their scalar rate on one core and six times it on the H100 under their transfer-inclusive convention, on every branch, while their scalar entry is faster than ours on every branch, by a third on the small-moneyness tile and by more than half in the wing. The vectorized port of the reference by Saqr [26] closes the picture from the iterative side. Its author reports the fused Triton kernel at 0.056 ms per 10^5 quotes on an H100 NVL, that is 0.56 ns per quote, CUDA-event compute time on a resident batch, at a worst relative error of 9×10^{-15} against the reference’s own C implementation. That is the author’s figure on a faster variant of the card and is not our measurement, since we have not run the kernel ourselves. Taken at face value it sits an order of magnitude behind the book kernel’s resident figures, 0.032 to 0.072 ns per quote, and seven times behind the routed charts’ bucket-ordered book at 0.078, which is the expected order, because a fused two-step Householder iteration on the GPU pays its four-branch masking and its error-function calls per lane, and the book kernel pays neither.

What survives is the conclusion worth stating. Reaching machine-precision accuracy costs nothing in throughput relative to a 10^{-7} table method, batch for batch, and accuracy here is not bought with speed. Glau et al. [9] build a Chebyshev interpolation of the implied-volatility surface as a single global object. Partitioning the domain at the inverse map’s branch points is what lets a low, fixed degree hold the last bit out to the boundary of feasibility rather than over a bounded box. Jäckel [14] remains the accuracy standard, and Table 2 shows the present method matches it everywhere both are at machine precision, while extending that precision into the two extreme regimes and changing the unit of work from a branchy scalar iteration to a vectorizable fixed-cost evaluation. The most recent work on the scalar side, Le Floc’h and Healy [17] and Le Floc’h [16], keeps that unit of work and optimizes inside it, with a cheap distribution-aware seed, one fast and two exact third-order Householder steps on an erfcx log-price objective, and no convergence loop, reaching 137 ns per call in Java against 243 for a Java port of the reference on their three-dimensional grid, at a worst error of 130 units in the last place of total volatility (23 with an optional final Newton polish, the reference’s own figure there). Their scalar latency

is of the same order as the book kernel’s scalar entry, 118 ns on our host against 194 for the reference in C++ at its release flags. The difference is that the book kernel reaches three units in the last place on that path and 28.6 ns per quote once eight quotes advance in lockstep, which a Householder chain, however trimmed, does not offer. A *vectorized* port of the reference does exist. Saqur [26] re-implements its seed, its normalised Black function and its two Householder corrections as whole-array operations with masked branch selection, in NumPy with Numba compilation on the CPU and as a single fused Triton kernel on the GPU. On our feed and host its Numba path (single thread, whole batch, from the raw pair) inverts at 411 ns per quote with a worst relative error of 3.3×10^{-14} against the book kernel’s reference, so the masking cost of running a four-branch iteration on every lane exceeds, on the CPU, what the width returns. Its author reports a Triton figure on an H100 which we return to with the GPU comparison of Section 7.

Limitations. Eight boundaries of the claims deserve collection in one place. First, every throughput comparison is against the reference’s official public scalar implementation, compiled at its author’s release flags into the same binary and timed by the same loop. The correct reading of Tables 5 and 4 is “faster than what a practitioner can obtain today,” not “faster than any conceivable vectorized iteration.” The one public vectorized port of the reference, measured on the CPU above, is slower than the scalar reference, and its GPU kernel we have not run. Second, the route mix (92% small-moneyness) is one year of one underlying under one tradeability screen. FX, single-name, commodity or short-dated books, and intraday quote streams, may distribute differently. Third, the batch speedups of Table 5 are largest on AVX-512. The AVX2 four-wide twins recover roughly half on the fixed-moneyness batches, though the market-feed advantage ($2.2\times$ against the release build) survives because every chart including the wing has an AVX2 twin, and hardware without SIMD falls back to the scalar path (still correct, still gated, still bit-identical, not accelerated). Fourth, the machine-precision domain has an explicit edge. The wing tables cover $\beta = h^2/16 \leq 16.4$, i.e. $h \leq 16.2$ (strikes beyond 10^7 times the forward), the rescue branch of Section 4.3 covers the band beyond it (validated for $W \lesssim 40$), and the still-deeper corner returns a flagged non-result. For $v > 8$ the returned volatility carries the input-representability floor of Section 3, which no double-precision interface can circumvent. Fifth, absolute timings are hardware-specific and a deployment decision should rest on a re-measurement on the target host. The accuracy and bit-identity results, which do not depend on timing conditions, are the load-bearing claims. Sixth, the exhaustive grid validation domain is $\sigma\sqrt{T} \leq 8$, $h \leq 16.5$. The wing evaluator is itself validated on prices to 10^{-320} , but the grid evidence of Figure 4 stops at the stated box. Seventh, the GPU figure of Table 6 is a kernel-resident, datacenter-fp64 measurement as qualified there, and any ratio formed against Table 5 compares hosts of different classes. The transportable facts are the per-quote device cost and the bitwise CPU–GPU identity, not a cross-host speedup factor. Eighth, the book kernel is a kernel for the traded book, not for the whole feasible box. On the uniform validation grid it answers one point in forty and the routed charts answer the rest, its reach in the wing ends at $h = 4$, and its device figure of 0.032 ns is for a book partitioned by the side of $a = 2\pi$, a partition that 2.2% of the nodes leave per day on the year’s data, and in file order warp divergence at its one branch costs a factor two.

Reproducibility. The oracle scripts, the accuracy and bit-identity harnesses, the benchmark driver and the frozen tables are public in the repository named in Appendix D, with build lines for all three instruction-set targets. Appendix D lists the artifacts. A golden-vector verifier (oracle-checked $(h, c) \rightarrow w$ triples spanning every chart and seam) lets an adopter confirm, in one command after building, that their compiler and hardware reproduce both the accuracy gate and the bit-identity invariant. Every number in this section is regenerated by the harnesses from the shipped sources.

8 Conclusion

Recovering implied volatility to machine precision is a solved problem for a single scalar quote. What has been missing is a method that reaches that precision through an evaluation that also vectorizes over the whole feasible domain, and the present paper supplies one in two stages. The first is a routed inverter. A branch-point-sized interior Chebyshev table and three matched endpoint charts, unified by a single two-parameter special-function family, resolve every feasible quote by routed evaluation with fixed-count local refinement, and branchless kernels shared between the scalar and SIMD paths make five distinct builds (two compilers, three instruction sets) return bitwise-identical doubles on every validation quote, extended to a sixth, architecturally alien build by the same kernels mirrored as CUDA device code, which invert a bucket-ordered book on one NVIDIA H100 at 0.078 ns per quote. The second stage removes the routing from the book. The coordinates in which both seams are level sets, $\theta = k/a_\rho$ and a_ρ itself, are intrinsic to the inverse rather than to any one chart, and in them the small-moneyness expansion is not fitted but exact, in that every coefficient beyond one tabulated function of a single variable is a rational number, and the same recurrence, re-expanded in a conformal variable adapted to the nearest singularity, reaches across the interior into the deep wing. One straight-line kernel with a 63-term table therefore covers all but three quotes in thirty thousand of the traded book, the routed charts standing behind it for the rare quote outside its reach and for the feasible domain a book never visits. Against a 40-digit oracle the worst relative error of the entry point so composed is 5.7×10^{-16} , 9.2×10^{-16} for the routed charts alone, held into the near-intrinsic and deep-wing regimes where the reference solver degrades, and every representable input receives a result of the validated accuracy class or a flagged non-result. In one binary with the reference at its author’s release flags the book kernel’s batch path inverts a market-realistic S&P 500 book at 6.7 times the reference’s rate on AVX-512 and 3.8 on AVX2, its scalar entry at 1.6 times, and the routed charts’ warm refinement of a persistent book at about five times under the few-ULP contract. On the H100 the exact-row kernel inverts the small-moneyness book at 0.021 ns per quote and the whole-book kernel, on a book sorted once by its intrinsic coordinate, the full book at 0.032. The nearest published relative of this design, the 10^{-7} polynomial table of Matić et al. [22], run from its authors’ code on the same card and the same quotes, branch by branch on identical tiles, is six times slower under its own transfer-inclusive convention and ten orders of magnitude less accurate on the traded book, its single-threaded scalar evaluation being the one figure it wins. The two methods together make the point that machine-precision accuracy is not bought with speed. Since all these solvers are at their design precision on tradeable quotes, the practical case is throughput and determinism, with the accuracy margin as insurance at the extremes. The natural extension is a Bachelier (normal-volatility) analogue, for which the same architecture applies with the normal density in place of the log-normal.

References

- [1] G. A. Baker and P. Graves-Morris. *Padé Approximants*. Cambridge University Press, Cambridge, 2nd edition, 1996. doi: 10.1017/CBO9780511530074.
- [2] O. Barndorff-Nielsen and C. Halgreen. Infinite divisibility of the hyperbolic and generalized inverse gaussian distributions. *Zeitschrift für Wahrscheinlichkeitstheorie und verwandte Gebiete*, 38(4):309–311, 1977.
- [3] F. Black and M. Scholes. The pricing of options and corporate liabilities. *Journal of Political Economy*, 81(3):637–654, 1973. doi: 10.1086/260062.
- [4] M. Brenner and M. G. Subrahmanyam. A simple formula to compute the implied standard deviation. *Financial Analysts Journal*, 44(5):80–83, 1988. doi: 10.2469/faj.v44.n5.80.
- [5] W. J. Cody. Rational Chebyshev approximations for the error function. *Mathematics of Computation*, 23(107):631–637, 1969. doi: 10.1090/S0025-5718-1969-0247736-4.
- [6] C. J. Corrado and T. W. Miller. A note on a simple, accurate formula to compute implied standard deviations. *Journal of Banking & Finance*, 20(3):595–603, 1996. doi: 10.1016/0378-4266(95)00014-3.
- [7] J. Demmel and H. D. Nguyen. Parallel reproducible summation. *IEEE Transactions on Computers*, 64(7):2060–2070, 2015. doi: 10.1109/TC.2014.2345391.
- [8] J. Gatheral. *The Volatility Surface: A Practitioner’s Guide*. John Wiley & Sons, Hoboken, NJ, 2006. ISBN 9780471792512.
- [9] K. Glau, P. Herold, D. B. Madan, and C. Pötz. The Chebyshev method for the implied volatility. *Journal of Computational Finance*, 23(3):1–31, 2019. arXiv:1710.01797.
- [10] D. Goldberg. What every computer scientist should know about floating-point arithmetic. *ACM Computing Surveys*, 23(1):5–48, 1991. doi: 10.1145/103162.103163.
- [11] P. S. Hagan, D. Kumar, A. S. Lesniewski, and D. E. Woodward. Managing smile risk. *Wilmott Magazine*, pages 84–108, 2002. September issue.
- [12] IEEE. IEEE standard for floating-point arithmetic, 2019. IEEE Std 754-2019.
- [13] P. Jäckel. By implication. *Wilmott*, pages 60–66, 2006. November issue.
- [14] P. Jäckel. Let’s be rational. *Wilmott*, 2015(75):40–53, 2015. doi: 10.1002/wilm.10395.
- [15] B. Jørgensen. *Statistical Properties of the Generalized Inverse Gaussian Distribution*, volume 9 of *Lecture Notes in Statistics*. Springer, New York, 1982.
- [16] F. Le Floc’h. Faster monotone implied volatility solver. arXiv:2605.22427, 2026.
- [17] F. Le Floc’h and J. Healy. Implying volatility: How fast can we go? arXiv:2605.29102, 2026.
- [18] R. W. Lee. The moment formula for implied volatility at extreme strikes. *Mathematical Finance*, 14(3):469–480, 2004. doi: 10.1111/j.0960-1627.2004.00200.x.
- [19] M. Li. Approximate inversion of the Black–Scholes formula using rational functions. *European Journal of Operational Research*, 185(2):743–759, 2008. doi: 10.1016/j.ejor.2006.12.028.

- [20] S. Manaster and G. Koehler. The calculation of implied variances from the Black–Scholes model: A note. *The Journal of Finance*, 37(1):227–230, 1982. doi: 10.1111/j.1540-6261.1982.tb01105.x.
- [21] J. C. Mason and D. C. Handscomb. *Chebyshev Polynomials*. Chapman & Hall/CRC, Boca Raton, FL, 2003.
- [22] I. Matić, R. Radoičić, and D. Stefanica. A PDE method for estimation of implied volatility. *Quantitative Finance*, 20(3):393–408, 2020. doi: 10.1080/14697688.2019.1675898. SSRN 3264356 (2018).
- [23] R. C. Merton. Theory of rational option pricing. *The Bell Journal of Economics and Management Science*, 4(1):141–183, 1973. doi: 10.2307/3003143.
- [24] J.-M. Muller, N. Brunie, F. de Dinechin, C.-P. Jeannerod, M. Joldes, V. Lefèvre, G. Melquiond, N. Revol, and S. Torres. *Handbook of Floating-Point Arithmetic*. Birkhäuser, 2nd edition, 2018. ISBN 9783319765259. doi: 10.1007/978-3-319-76526-6.
- [25] F. W. J. Olver, D. W. Lozier, R. F. Boisvert, and C. W. Clark. *NIST Handbook of Mathematical Functions*. Cambridge University Press, New York, 2010.
- [26] R. Saqur. Fast-vollib: A fast implied volatility library for Python with PyTorch, JAX, and CUDA fused-kernel backends. arXiv:2604.27210; <https://github.com/raeidsaqur/fast-vollib>, 2026.
- [27] W. Schadner. Implied volatility as a quantile. *Quantitative Finance*, 2026. doi: 10.1080/14697688.2026.2721562. forthcoming.
- [28] W. Schadner. Moneyness as a global projective modulus of the Black–Scholes inverse. Technical Report 7462078, SSRN Electronic Journal, 2026. URL https://papers.ssrn.com/sol3/papers.cfm?abstract_id=7462078.
- [29] D. Stefanica and R. Radoičić. An explicit implied volatility formula. *International Journal of Theoretical and Applied Finance*, 20(7):1750048, 2017. doi: 10.1142/S0219024917500480.
- [30] L. N. Trefethen. *Approximation Theory and Approximation Practice*. SIAM, Philadelphia, extended edition, 2019. doi: 10.1137/1.9781611975949.
- [31] Y. Xia and Z. Cui. An exact and explicit implied volatility inversion formula. *International Journal of Financial Engineering*, 5(3):1850032, 2018. doi: 10.1142/S2424786318500329.

Appendix

A Engineering of the matched chart and the tabulated chart

The two charts of Sections 4.1 and 4.2 that the book kernel of Section 5 replaces on the traded book remain the inverter’s coverage outside the kernel’s reach, and their engineering is recorded here in the form the runtime executes.

The matched small-moneyness chart. The chart is completed by two implementation decisions. First, the residual R is small enough to be frozen once as a single low-order bivariate Chebyshev surface (degree two in the h^2 direction, sixteen in s). It absorbs everything beyond h^4 and brings the chart to machine precision across its whole strip. Second, the two functions V_2 and V_4 , whose closed forms contain exponentials and a series crossover, are replaced by one-dimensional Chebyshev expansions in s of degrees 28 and 26. This removes the last library call and the last data-dependent branch from the chart’s inner loop, which matters for the vectorized path of Section 6. The universal inverse B^{-1} itself is evaluated by a three-regime branchless Chebyshev approximation, one regime in the variable $u = \kappa/(\rho + \frac{1}{2})$ for moderate arguments and two in $L = \log(1/\rho)$ for the tail, totalling 456 bytes of coefficients with worst error 5.4×10^{-16} . The chain $c \mapsto \rho \mapsto A \mapsto s$ has relative condition number at most one, so no accuracy is lost in the transforms.

The tabulated chart: degrees, cells and bytes. The surrogate (7) is sized as follows. The degree never exceeds sixteen in the price direction and thirteen in the moneyness direction, with median near ten. The full table comprises nineteen h -bands over $h \in [0.30, 6.65]$ and 181 cells holding about twenty-two thousand double-precision coefficients, just under 200 KB in total, small enough to be resident in a single core’s L2 cache, so that in steady state the table costs no memory traffic. Cache-cold and cache-warm evaluation are indistinguishable after the first few hundred quotes of a batch, so residency is realized in practice. Recovery from the surrogate is the single map $w = h^2/(2W)$.

Why the fitted degrees sit below the geometric rate. Two things sharpen that account. The first concerns the vertical offset by which the fitted degrees sit below the predicted geometric rate in Figure 3(b), which we described above as an unknown order-one prefactor. It is not unknown. When the dominant singularity is a power of a logarithm, which is the end type of this inverse at both price endpoints, the Chebyshev coefficients behave as $a_n \sim C \varrho^{-n} n^{-1} (\log n)^{q-1}$ with $q = \frac{1}{2}$ at an outer end and $q = \frac{3}{2}$ at an inner one. Over the degree range the table actually uses, $n \leq 16$, a straight-line fit of $\log |a_n|$ absorbs that algebraic and logarithmic prefactor as a roughly constant vertical shift, which is what the panel shows, and the degree rule is correspondingly the fixed point of $N \approx [\log(1/\varepsilon) - \log N - q \log \log N] / \log \varrho$ rather than a pure geometric rate. The second concerns the design principle itself, which moves partway from assumption to theorem. In the variable $L = -\log \rho$, an affine reflection of λ , the physical price branch at fixed k is single-valued and holomorphic on the half-strip $\Re L > \log(e^k - 1)$, $|\Im L| < \pi$, and the finite set of prices at which the branch can fail is not unstructured but the discrete lattice $\{L_0 + 2\pi i n\} \cup \{(2n+1)\pi i\} \cup \{\ell_0 + (2n+1)\pi i\}$, with $L_0 = \log(e^k - 1)$ and $\ell_0 = L_0 - k$. Inside the strip the nearest obstruction is therefore the price endpoint, exactly as assumed, and that settles the cells nearest the money, whose Bernstein ellipses stay within it and which are also the cells setting the worst case $\rho = 3 + 2\sqrt{2}$ drawn in panel (a). For the wider cells, whose ellipses have semi-minor axis exceeding π , what remains open is no longer where the obstructions are, since the lattice locates them, but whether the physical sheet continues past them. The fitted degrees indicate that it does.

Routing to a cell. Routing to a cell is performed by integer operations on the IEEE-754 bit fields of the inputs, before any transcendental evaluation. The high bits of h (exponent plus two leading mantissa bits) select the h -band. The exponent bits of c select a price octave within the band, and the mantissa of c , forced into $[1, 2)$, becomes the evaluation abscissa through the shared logarithm kernel of Section 6, so that the per-quote classification involves no division, no logarithm and no comparison ladder. The octaves adjacent to $c = 1$, where the near-at-the-money geometry would otherwise demand a special analytic branch, are instead refined by one or two additional mantissa bits into sub-cells, so that they become ordinary table cells. This sub-splitting is what removes the last analytic bypass from the interior.

The extent of the bands. The bands deliberately stop at $H_{\text{box}} \approx 6.65$. Beyond it the feasible strip is owned entirely by the endpoint charts of Sections 4.1, 4.3 and 4.4. For such large h every feasible quote is either deep in the wing (small price) or above the $v = 1.85$ ceiling (large volatility), so extending the table would add cells that no quote needs. In fact the handover happens earlier still. The two seams cross at $h = 5.10$, beyond which the ceiling lies below the wing seam and no price can reach the table at all. The same reasoning caps the tabulated maturity range from below at $h = 0.3$, where the matched small-moneyness chart is uniformly accurate. The table is therefore exactly as large as the region in which neither endpoint structure simplifies, and no larger.

Generation. The table is generated offline and oracle-gated before it is frozen. Chebyshev nodes for each cell are obtained by inverting exact double-precision prices with an arbitrary-precision solver at no fewer than twenty-five significant digits. The tensor coefficients are fitted, and the cell is accepted only if an *independent* forty-digit oracle, distinct from the generator, confirms a relative σ error below 10^{-15} on a dense sample of the cell including its boundaries. Cells failing the gate are re-fitted at higher degree or split. The same oracle-gated discipline applies to every auxiliary kernel of Section 6.

B The hot path

The complete scalar evaluation, which the batch drivers reproduce bucket-wise, is the following. All constants are frozen doubles. $\text{bits}(\cdot)$ denotes IEEE-754 bit extraction, and $\log_2^*$ is the shared logarithm kernel of Section 6.

Routing (given $h \geq 0$ and $c \in (0, 1)$):

1. if $h = 0$: solve the exact ATM relation $c = \text{erf}(\sigma/2\sqrt{2})$ (quantile seed, two guarded corrections). Return.
2. if $c < c_w(h)$ (one Chebyshev in h): WING.
3. if $h < 0.3$: NEAR if $c \leq c_{\text{top}}(h)$ (a frozen Chebyshev fit of $C(h, 1.85)$, so the test is a polynomial, not a price evaluation), else UPPER.
4. if $h \leq H_{\text{box}}$ and $c \leq c_{\text{top}}(h)$: FAR. Else UPPER.

Both seam polynomials are evaluated for every quote, whichever chart it reaches, and the order of the tests above is immaterial to cost. Each tabulated chart is a *band*, bounded below by c_w and above by c_{top} , so establishing membership requires both boundaries however the branches are arranged, and the comparisons are negligible beside the two Chebyshev sweeps that produce them. This is deliberate rather than incidental: the vectorized classifier evaluates both seams on every lane and selects by mask, since a branch that skipped work for some lanes would break the lockstep on which bit-identity rests (Section 6). The one place the empirical route mix is exploited is the choice of batch driver, not the branch order—when a sampled probe finds the

small-moneyness share high, the speculative driver evaluates NEAR on all lanes unconditionally and defers the minority.

NEAR: $\rho = c / \expm1(h)$; $A = B^{-1}(\rho)$ (three-regime Chebyshev); $s = h/A$; $v = v_0(\kappa s) + h^2 V_2(s) + h^4 V_4(s) + h^6 R(s, h)$. Return $w = v^2$.

NEAR, intrinsic form (Section 5): $E = \expm1(h)$; $a = \log 1p(E/c)$ (Sterbenz-exact reduction); $A_0 = a G(a)$ (25-term Clenshaw); $t = h/A_0$, $x = t^2$, $y = A_0^2$; $v = t \sum_{m \leq 10} x^m P_m(y)$ (55 exact fma). Return $w = v^2$. Above $\tau = 0.035$: the matched form. Whole-book form (the entry point as shipped): seventeen rows for $a < 2\pi$ and $h \leq 0.35a$; $1/A_0$ from the second cell, $u F_1(u)$ with $u = 1/\sqrt{a}$, for $a \geq 2\pi$, and there $v = t(1 + \sum_{n \leq 18} q^n d_n(1/A_0^2))$ with $q = z/(\sqrt{1+z} + 1)^2$, $z = h^2/4\pi^2$, for $h \leq \min(4, 0.16a + 0.85)$. Outside both: the routed charts below.

FAR (table): band from $\text{bits}(h)$. Octave and sub-cell from $\text{bits}(c)$; $x_\lambda = \alpha_{\text{cell}} \log_2^*(m) + \beta_{\text{cell}}$ with m the mantissa of c ; W by the bivariate Clenshaw sweep (7). Return $w = h^2/(2W)$.

WING: if $h^2/16 > 16.4$: rescue branch (Φ -inversion seed, three fixed-point corrections, three Householder steps. Quiet NaN beyond its validated band). Otherwise $\tilde{L}$ from $\log c$ (subnormal-safe $\log_2^*$) and h . Regime (piece A / piece B / series) once from $\tilde{L}$. Seed $W_0 = \max(\tilde{L} - \frac{3}{2} \log \tilde{L}, 2.6)$, plus the degree-(8,8) peeled-residual surface in $(\tilde{L}, \beta)$ for the fitted pieces (bare W_0 in the series regime). Exactly three Newton steps on $W = \tilde{L} - \frac{3}{2} \log W + \log H(W)$ with $\log H$ and its analytic derivative from the fitted $S(u, \beta)$ (or the series). Return $w = h^2/(2W)$.

UPPER: $\bar{g} = \frac{1}{2}(1-c)e^{-h/2}$; $x_0 = -\Phi_{\text{seed}}^{-1}(\bar{g})$ (rational seed); Mills ratio $m_0 = \sqrt{\pi/2} \operatorname{erfcx}(x_0/\sqrt{2})$. Ladder corrections $x = x_0 + h^2 x_2 + h^4 x_4$ with x_2, x_4 polynomial in $1/x_0, m_0$. Then exactly three Householder-3 steps on (11), with f/g' from (13) and the closed-form ratios $g''/g' = -x + h^2/(4x^3)$, $g'''/g' = (g''/g')^2 - 1 - 3h^2/(4x^4)$, prices evaluated in whichever of $c, 1-c$ is small. Return $w = (2x)^2$.

Batch drivers: a strided moneyness sample selects one of two paths. *Speculative* (small-moneyness-heavy feeds, the live case): stream eight/four-wide. Classify each lane by the frozen seam polynomials. Evaluate the small-moneyness kernel on every lane. Masked-store the small-moneyness lanes in input order. Compress the rest into a deferred buffer. *Sort-then-batch* (the deferred buffer, and minority-dominated feeds): classify, counting-sort by cell, chart and wing regime, and evaluate each bucket eight/four-wide. Both are bit-identical to the scalar entry, so the dispatch changes only speed. Remainders, the rescue branch and the ATM line go through the scalar entry unchanged.

C Table generation and validation

The offline pipeline that produces the frozen constants is as important to the accuracy claim as the runtime, so we record it.

1. *Truth generation.* For each cell, Chebyshev nodes in (x_h, x_λ) are mapped to exact double-precision prices, and each price is inverted by an arbitrary-precision solver (no fewer than 25 significant digits, price residual driven below 10^{-18}). The transformed truth $W = h^2/(2w)$ is recorded.
2. *Fitting.* Tensor Chebyshev coefficients are computed per cell. Degrees are the smallest passing the gate below, capped at (13, 16).
3. *Gating.* An independent 40-digit oracle, sharing no code with the generator, re-inverts a dense sample of each cell including its boundaries and the sub-cell seams. The cell is accepted only if the relative σ error is below 10^{-15} at every sample. Failing cells are refit at higher degree or split. The same gate validates every auxiliary kernel ($\log_2^*$, v_0 , B^{-1} , V_2 , V_4 , erfcx , the exponential, the residual R), with the additional downstream-degree criterion of Section 6.

4. *Freezing.* Accepted coefficients are emitted as exact hexadecimal doubles into a header. The runtime contains no fitted constant that did not pass the gate.

Certified enclosures for the endpoint functions are available independently of the gate through the Stieltjes representation of Section 4.5, whose continued-fraction convergents bound B , B_n and B^{-1} from alternating sides.

For the benefit of implementers we also record the design alternatives that the gate *rejected*, since each looks attractive on paper. A transcendental-free table in the raw mantissa coordinate (routing and abscissa both from bit fields, no logarithm at all) fails on degree. The h -independent branch-point distances of Section 4.2 pin the Bernstein parameter and roughly double the degree per cell, blowing the byte budget. A conformal coordinate adapted to the annulus fails the same way, more severely. Evaluating the cells in the monomial basis with Estrin’s scheme, instead of Chebyshev with Clenshaw’s, fails the precision gate outright at the required degrees. An equal-accuracy minimax replacement for the logarithm kernel fails through the downstream-degree effect described in Section 6. Fitting the wing’s $\log H$ directly, or its unweighted peeled residual, demands degrees past thirty-six where the σ -weighted form needs twenty-six, and a $(1/W, \beta/W)$ chart for the same fit, natural-looking because $\beta/W = v^2/8$ is bounded, brings no degree reduction because the hard structure at the small- W edge is genuinely asymptotic rather than an artifact of coordinates. Finally, one accepted choice had to be revised by a denser gate. Five wing Newton steps passed a sparse step-count de-risk yet left six points of the dense regular grid above 10^{-15} (worst 1.2×10^{-14}), all at the high- β , low- W corner where convergence is slowest. With the bare logarithmic seed the safe fixed count is six, and the incident is a reminder that step-count de-risking must sample the corner of slowest convergence, not the typical point. The tabulated seed of Section 4.3 later cut the count to three, and that reduction was gated on a dense sweep of the same corner plus the series regime, per the lesson. Each rejection was measured, not argued.

D Artifacts

The implementation and every harness behind the tables are public in the repository <https://github.com/wol-fi/volfi> (release v0.3.0, BSD 3-Clause). It contains the header-only C++17 implementation (the book kernel with its 63-term table and exact rows, its SIMD twins and the generator that derives the rows in exact rational arithmetic, and the routed charts’ runtime, frozen tables, wing evaluator and checked API). The oracle generators (`mpmath`, 40 digits) for the regular, stressed and edge sets and for the boundary-concentrated campaign of the book kernel, with the 60-digit recheck of its worst points, and the node-persistence measurement over the year’s consecutive snapshots. The golden-vector verifier and its oracle files (the one-command build check of Section 7). The accuracy harness producing Table 2, Table 3 and Figure 4, and the one-binary harness producing Table 4, with the build lines that compile the reference at its author’s flags with and without link-time optimization. The bit-identity and poison-test harnesses. The throughput benchmark, including the build lines for the AVX-512, AVX2 and scalar targets used in Section 7, and the raw outputs of the quiet-host campaign. The CUDA full-book port behind Table 6, comprising the device kernels mirrored from the scalar entry, the bucket-ordered driver with its per-chart ULP cross-check against the CPU reference compiled into the same binary, the mechanical device-constant generator, and the `nvcc` build line (`-arch=sm_90 -fmad=false`). The device driver of the intrinsic kernels of Section 5, with its host self-check, its sorted and transfer-inclusive timings and its region tiles. Finally, the harness that builds the table method of Matić et al. [22] from its authors’ published sources and times it on the same feed and tiles, on the CPU and through its OpenCL path (Table 7), with the container recipe that supplies the OpenCL loader the CUDA image lacks.

Three inputs are not redistributed, because their licences are not ours to pass on. The

market feed derives from a licensed OptionMetrics file, so the repository carries the filter and projection recipe needed to regenerate the 30,000 normalized (h, c) pairs from a licensed copy, and every harness that consumes the feed accepts any file of such pairs; the verification suite and every accuracy table run without it. The reference implementation of Jäckel [14] (revision 1520, obtained from <http://www.jaeckel.org/>) is the version current when the benchmarks were run; the harnesses compile it unmodified from the author’s sources at a path the caller supplies. The PDE table method is built from its authors’ MIT-licensed sources and their 46 MB coefficient file, both obtained from their repository as its instructions describe. For independent audit, the closed forms of V_2 and V_4 , the ladder coefficients B_n , the frozen seam and wing fits, and the exact residual equations driving the Householder and Newton refiners appear in the shipped headers in the notation of the text. One portability note deserves record. A code-generation defect in one Windows GCC toolchain (misaligned vector spills in a heavily spilled function, affecting AVX-512 and AVX2 builds alike and unrelated to this code’s logic) can crash the mixed-grid driver on that platform. The artifacts document the affected configuration, and all results here are from Linux, which is unaffected.

Workload	quotes	ns per quote
NEAR surface, $h=0.2$	5,242,880	0.038
FAR surface, $h=1$	5,242,880	0.140
WING surface, $h=1$	5,242,880	0.669
UPPER surface, $h=1$	5,242,880	0.168
Full book (2024 SPX market mix)	5,010,000	0.078
recurrence kernel, NEAR feed tile, file order or sorted	4,990,713	0.021
table kernel of Section 5, NEAR feed tile	4,990,713	0.036
whole-book kernel, full feed, file order	4,980,000	0.072
whole-book kernel, full feed, sorted by a	4,980,000	0.032
whole-book kernel, sorted, with transfers [‡]	4,980,000	1.12
PDE table method [22], with transfers [‡]	4,979,968	7.68

Table 6: GPU throughput of the bucket-ordered coalesced schedule on one NVIDIA H100 (PCIe, CUDA 12.4, `-fmad=false`), median nanoseconds per quote over 300 timed passes, two independent runs agreeing to three digits on every row but the tabulated surface, where they agree to two (0.1385 and 0.1414). Every result is bitwise identical to the CPU scalar entry (zero mismatches at zero ULP, on every row). Rows follow the chart order of Section 4 and of Table 5. The four chart rows are chart-pure fixed-moneyness surfaces carrying the same workloads as the corresponding rows of Table 5, each quote route-filtered through the classifier the market driver itself uses. Unlike the CPU, the device runs every chart as one kernel over a contiguous slice whether moneyness is fixed or mixed, so no row here is measured through a different execution path than the others. A synthetic surface is also what makes the large-volatility row measurable at all, since that chart receives no quotes on this feed, as is likewise true of the scalar-only branches (rescue wedge, analytic edges). The full-book row is the market feed itself, at its live route mix of 91.9% small-moneyness, 7.0% tabulated and 1.1% deep wing, and in that setting the three charts cost 0.057, 0.153 and 1.167 respectively. The wing is far dearer in situ than on its fixed-moneyness surface because the routed wing quotes span a wider range of h and sit deeper, the more so since the seam of Section 4 hands all but the genuinely extreme quotes to the tabulated charts. The lower block is the intrinsic-chart session of Section 5 on the same host class (H100 PCIe, CUDA 12.4, `-fmad=false`, 300 passes, two runs agreeing to three digits), with the feed tiled to about five million quotes and every kernel bit-identical to the CPU entry on every quote (4,979,502 covered quotes for the whole-book kernel, 498 handed to the routed charts). The whole-book kernel’s file-order and sorted figures differ by a factor two because its one branch, at $a = 2\pi$, diverges inside mixed warps. [‡]Uploads h and c from pageable host memory and reads w back on every pass, the convention of the PDE method’s published timing. That method’s row is its OpenCL double-precision path built from the authors’ sources, with their 46 MB table, on the same card and the same quotes.

Region	feed quotes	PDE method	PDE method, CPU		Book kernel, CPU		PDE, H100	Book kernel, H100	
		max rel err	scalar	OpenMP	scalar	batch	with transfers	resident /	with transfers
NEAR	27,574	1.2×10^{-5}	76	—	115	25	7.09	0.070 /	1.14
FAR	2,084	1.7×10^{-8}	65	—	135	38	6.77	0.087 /	1.16
WING	342	5.3×10^{-9}	67	—	174	39	6.71	0.045 /	1.12
whole feed	30,000	1.2×10^{-5}	76	11*	118	29	7.68	0.032 /	1.12

Table 7: The PDE table method of Matić et al. [22] and the book kernel on identical region-filtered tiles of the market feed, ns per quote unless stated. Accuracy is the worst relative error in σ against the book kernel, itself within 3 units in the last place of the 40-digit oracle on this feed. The book kernel’s own worst error on the feed is 4.9×10^{-16} on every branch. CPU columns are one thread on the quiet host of Table 5, AVX-512 build, tiles of 30,000 (*their OpenMP column, their published convention, is all 28 cores of the H100 host’s CPU). The device tiles cycle each region’s feed quotes to 4,980,000. “With transfers” uploads the inputs and reads the results back on every pass, the convention of their published timing and their wrapper’s only mode. The book kernel’s resident figure is the kernel alone on a book already on the device, sorted by a for the whole-feed row and in file order for the region tiles.